

Numéro d'ordre :

Mémoire présenté en vue de l'obtention de Diplôme de Licence En Sciences et Techniques

Prévision Statistique de la Volatilité et Allocation Dynamique de Portefeuille par Apprentissage.

Filière : Mathématiques et informatique (MI)

Département : Mathématiques

Présenté à la Faculté des Sciences et Techniques Al Hoceima par

EL HABNOUNI Safwan

Le 24 juin 2026

Devant le jury

Pr. Darhouche Omar	Faculté des Sciences et Techniques Al Hoceima	President
Pr. Moradi Najat	Faculté des Sciences et Techniques Al Hoceima	Examinatrice
Pr. Hafid Abdelatif	Faculté des Sciences et Techniques Al Hoceima	Encadrant

DÉDICACE

À mes très chers parents,

*pour leur amour inconditionnel, leurs
sacrifices
et leurs prières.*

À mes frères et à ma sœur,

*pour leur présence bienveillante
et leurs encouragements constants.*

À mes amis et à mes camarades,

pour ces années d'études inoubliables.

REMERCIEMENTS

Avant tout, je rends grâce à **Allah Le Tout-Puissant** qui m'a donné la force, la patience et la volonté nécessaires pour mener à bien ce travail. Sa grâce m'a accompagné tout au long de ce parcours.

Au terme de ce projet de fin d'études, je tiens à exprimer ma sincère gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce travail.

Je remercie particulièrement mon encadrant, **Professeur Abdelatif Hafid**, pour son encadrement, sa disponibilité, ses conseils précieux et son accompagnement tout au long de ce projet. Ses orientations et son soutien ont grandement contribué à la réussite de ce travail.

J'adresse également mes remerciements aux membres du jury, **Professeur O. Darhouche**, président du jury, et **Professeur N. Moradi**, examinateur de ce travail. Je leur suis reconnaissant pour l'intérêt accordé à mon projet et pour l'honneur qu'ils me font en l'évaluant.

Je tiens à remercier l'ensemble des **professeurs de la Faculté des Sciences et Techniques d'Al Hoceima** pour la qualité de leur enseignement, leur accompagnement et les connaissances transmises durant ces années de formation.

Mes remerciements les plus sincères vont à **mes chers parents**, pour leur soutien, leurs encouragements et leurs sacrifices qui ont toujours été une source de motivation. Je leur dédie ce travail avec toute ma gratitude.

À **mes frères, ma sœur et toute ma famille**, merci pour votre présence, votre soutien et vos encouragements tout au long de ce parcours.

Enfin, je remercie chaleureusement **mes amis et camarades** pour les bons moments partagés, leur entraide et leur soutien durant ces années d'études.

Merci à toutes et à tous.

RÉSUMÉ

Dans un contexte financier mondial marqué par une volatilité croissante et des marchés de plus en plus imprévisibles, la capacité à anticiper le risque est devenue un enjeu majeur pour les investisseurs et les gestionnaires de portefeuille. Ce projet de fin d'études s'inscrit dans cette problématique en proposant une approche quantitative combinant des techniques d'apprentissage automatique (*Machine Learning*) et des méthodes d'optimisation financière pour la prévision de la volatilité et l'allocation dynamique de portefeuille.

Les données utilisées proviennent des historiques boursiers de deux actifs complémentaires : **SPY** (ETF répliquant le S&P 500, représentant le marché stable) et **TSLA** (l'action Tesla, représentant un actif à haute volatilité), couvrant une période de six années allant de **janvier 2018 à janvier 2024**. Ces données ont été téléchargées via la bibliothèque Python `yfinance` depuis Yahoo Finance.

La variable cible du projet est la **volatilité empirique**, calculée comme l'écart-type des rendements journaliers sur une fenêtre glissante de 20 jours. L'ingénierie des variables (*feature engineering*) a permis de construire un ensemble de prédicteurs riches incluant les rendements des cinq derniers jours, l'historique de la volatilité, ainsi que deux indicateurs techniques mathématiques : le **RSI** (*Relative Strength Index*) sur 14 jours et le **MACD** (*Moving Average Convergence Divergence*).

Trois modèles de régression ont été entraînés, évalués et comparés : la **Régression Linéaire Multiple** (MCO), la **Régression Ridge** (régularisation L2) et le **Random Forest Regressor**. Un **modèle d'Ensemble Learning** combinant les deux meilleurs modèles via un *Voting Regressor* a ensuite été développé pour améliorer davantage les prédictions.

Les prédictions de volatilité ont été exploitées pour construire une **stratégie d'allocation dynamique de portefeuille** basée sur la parité des risques (*Inverse Volatility Weighting*). Les performances de cette stratégie ont été évaluées à l'aide de trois ratios financiers standards : le **Ratio de Sharpe**, le **Ratio de Sortino** et le **Ratio de Calmar**.

Les résultats obtenus démontrent la supériorité du modèle d'Ensemble Learning sur les modèles individuels, et la stratégie d'allocation dynamique pilotée par l'intelligence artificielle surpasse significativement la stratégie naïve d'investissement équipondéré (50%/50%).

Mots-clés : Volatilité financière, Apprentissage automatique, Régression Ridge, Random Forest, Ensemble Learning, Allocation de portefeuille, Parité des risques, Ratio de Sharpe, RSI, MACD.

ABSTRACT

In a global financial context characterized by increasing volatility and ever more unpredictable markets, the ability to anticipate risk has become a major challenge for investors and portfolio managers alike. This final-year project addresses this issue by proposing a quantitative approach that combines Machine Learning techniques with financial optimization methods for volatility forecasting and dynamic portfolio allocation.

The data used in this project comes from the historical stock records of two complementary assets : **SPY** (an ETF tracking the S&P 500, representing the stable market) and **TSLA** (Tesla stock, representing a high-volatility asset), covering a six-year period from **January 2018 to January 2024**. This data was downloaded using the Python library `yfinance` from Yahoo Finance.

The target variable of the project is **empirical volatility**, computed as the standard deviation of daily returns over a rolling 20-day window. Feature engineering was applied to build a rich set of predictors, including returns from the past five days, historical volatility, and two mathematical technical indicators : the **RSI** (*Relative Strength Index*) over 14 days and the **MACD** (*Moving Average Convergence Divergence*).

Three regression models were trained, evaluated and compared : **Multiple Linear Regression** (OLS), **Ridge Regression** (L2 regularization), and the **Random Forest Regressor**. An **Ensemble Learning model** combining the two best-performing models via a *Voting Regressor* was subsequently developed to further improve predictions.

The volatility predictions were then used to build a **dynamic portfolio allocation strategy** based on risk parity (*Inverse Volatility Weighting*). The performance of this strategy was evaluated using three standard financial ratios : the **Sharpe Ratio**, the **Sortino Ratio**, and the **Calmar Ratio**.

The results obtained demonstrate the superiority of the Ensemble Learning model over individual models, and the AI-driven dynamic allocation strategy significantly outperforms the naive equally-weighted investment strategy (50%/50%).

Keywords : Financial volatility, Machine Learning, Ridge Regression, Random Forest, Ensemble Learning, Portfolio allocation, Risk parity, Sharpe Ratio, RSI, MACD.

ملخص

في ظل سياق مالي عالمي يتسم بتزايد التقلبات وصعوبة التنبؤ بمسار الأسواق، أصبحت القدرة على استباق المخاطر تحدياً جوهرياً للمستثمرين ومديري المحافظ الاستثمارية. يندرج مشروع التخرج هذا ضمن هذا الإطار، حيث يقترح مقارنة كمية تجمع بين تقنيات التعلم الآلي (Learning Machine) وأساليب التحسين المالي، بهدف التنبؤ بالتقلبات والتوزيع الديناميكي للأصول داخل المحفظة.

تعتمد البيانات المستخدمة في هذه الدراسة على السجلات التاريخية لأسعار أصلين ماليين متكاملين: SPY (صندوق استثمار متداول يتبع مؤشر S&P 500، ويمثل السوق المستقر)، و TSLA (سهم شركة تسلا، ويمثل أصلاً عالي التقلب)، تغطي فترة ست سنوات تمتد من يناير 2018 إلى يناير 2024. وقد تم استخراج هذه البيانات عبر مكتبة بايثون yfinance من منصة Finance. Yahoo.

يمثل التقلب التجريبي المتغير المستهدف في هذا البحث العلمي، ويُحسب على أساس الانحراف المعياري للعوائد اليومية خلال نافذة زمنية متحركة مدتها 20 يوماً. علاوة على ذلك، مكنت هندسة الخصائص (engineering feature) من بناء مجموعة غنية من المتغيرات التنبؤية، شملت عوائد الأيام الخمسة الماضية، والتاريخ السابق للتقلبات، إلى جانب مؤشرين فنيين رياضيين هما: RSI (Index Strength Relative) المحسوب على مدار 14 يوماً، و MACD (Divergence Convergence Average Moving).

في مرحلة النمذجة، تم تدريب وتقييم ومقارنة ثلاثة نماذج انحدار: الانحدار الخطي المتعدد، (MCO) وانحدار ريدج، ونموذج الغابة العشوائية. ولتحسين دقة التنبؤات بشكل أكبر، تم تطوير نموذج تعلم مُجمّع (Learning Ensemble) يدمج أفضل نموذجين باستخدام تقنية الانحدار التصويتي (Regressor Voting).

أخيراً، تم توظيف تنبؤات التقلبات لبناء استراتيجية توزيع ديناميكي للمحفظة تركز على مبدأ تكافؤ المخاطر (Inverse Weighting Volatility). ولتحديد مدى نجاعة هذه الاستراتيجية، جرى تقييم أدائها بالاعتماد على ثلاثة مؤشرات مالية قياسية: نسبة شارب، ونسبة سورتينو، ونسبة كالمار.

تظهر النتائج المحصل عليها تفوق نموذج التعلم المُجمّع على النماذج الفردية، كما أن استراتيجية التوزيع الديناميكي الموجهة بالذكاء الاصطناعي تتفوق بشكل ملحوظ على استراتيجية الاستثمار التقليدية متساوية الأوزان (50%/50%).

الكلمات المفتاحية : التقلبات المالية، التعلم الآلي، انحدار ريدج، الغابة العشوائية، التعلم المُجمّع، توزيع المحفظة، تكافؤ المخاطر، نسبة شارب، RSI، MACD.

LISTE DES ABRÉVIATIONS

Abréviation	Signification
ML	<i>Machine Learning</i> — Apprentissage Automatique
MSE	<i>Mean Squared Error</i> — Erreur Quadratique Moyenne
RMSE	<i>Root Mean Squared Error</i> — Racine de l'Erreur Quadratique Moyenne
MAE	<i>Mean Absolute Error</i> — Erreur Absolue Moyenne
OLS	<i>Ordinary Least Squares</i> — Méthode des Moindres Carrés Ordinaires
MCO	Méthode des Moindres Carrés Ordinaires
GD	<i>Gradient Descent</i> — Descente de Gradient
SGD	<i>Stochastic Gradient Descent</i> — Descente de Gradient Stochastique
RF	<i>Random Forest</i> — Forêt Aléatoire
RSI	<i>Relative Strength Index</i> — Indice de Force Relative
MACD	<i>Moving Average Convergence Divergence</i>
EMA	<i>Exponential Moving Average</i> — Moyenne Mobile Exponentielle
MME	Moyenne Mobile Exponentielle
SPY	ETF répliquant l'indice boursier S&P 500
TSLA	Ticker boursier de l'action Tesla Inc.
ETF	<i>Exchange-Traded Fund</i> — Fonds Indiciel Coté en Bourse
PFE	Projet de Fin d'Études
IA	Intelligence Artificielle

Table des matières

1	Introduction Générale	2
1.1	Contexte Général	2
1.2	Problématique	3
1.3	Objectifs du Projet	4
1.4	Données Utilisées	5
1.5	Organisation du Mémoire	6
2	Préparation et Ingénierie des Données	9
2.1	Introduction	9
2.2	État de l’Art — Approches de Prédiction de la Volatilité	10
2.2.1	Modèles Statistiques Classiques	10
2.2.2	Apprentissage Automatique Appliqué à la Finance	11
2.2.3	Positionnement de Notre Approche	11
2.3	Présentation des Données	12
2.3.1	Source des Données : Yahoo Finance et <code>yfinance</code>	12
2.3.2	Actifs Sélectionnés	12
2.3.3	Période d’Étude et Justification	13
2.4	Calcul des Rendements Journaliers	14
2.4.1	Définition et Formule Mathématique	14
2.4.2	Pseudocode : Téléchargement et Calcul des Rendements	15
2.5	Construction de la Variable Cible : La Volatilité Empirique	16
2.5.1	Définition Mathématique	17
2.5.2	Justification du Choix de la Fenêtre de 20 Jours	17
2.5.3	Pseudocode : Calcul de la Volatilité Glissante	17
2.6	Ingénierie des Variables Explicatives (Feature Engineering)	19
2.6.1	Variables de Mémoire Courte : Les Rendements Passés	19
2.6.2	La Volatilité Historique Décalée	19
2.6.3	Le RSI (Relative Strength Index)	20
2.6.4	Le MACD (Moving Average Convergence Divergence)	21
2.6.5	Prévention du Data Leakage	23
2.7	Construction du Jeu de Données Final	24
2.7.1	Récapitulatif des Variables	24

2.7.2	Pseudocode : Construction du Jeu de Données Final	25
2.8	Séparation Temporelle Train / Test	25
2.8.1	Pourquoi Pas de Split Aléatoire en Finance ?	25
2.8.2	Le Split Temporel Strict 80% / 20%	26
2.8.3	Pseudocode : Split Temporel	27
2.9	Synthèse : Pipeline Complet de Préparation des Données	27
2.10	Conclusion	28
3	Méthodologie : Modèles et Concepts Mathématiques	31
3.1	Introduction	31
3.2	La Régression Linéaire Multiple	32
3.2.1	Formulation du Modèle	32
3.2.2	La Fonction de Coût : l'Erreur Quadratique Moyenne	33
3.3	Algorithmes d'Optimisation : Descente de Gradient	33
3.3.1	Principe de la Descente de Gradient	34
3.3.2	Pseudocode : Descente de Gradient (Batch)	34
3.3.3	La Descente de Gradient Stochastique (SGD)	35
3.3.4	Pseudocode : Descente de Gradient Stochastique	36
3.4	La Régression Ridge (Régularisation L2)	37
3.4.1	Motivation : Pourquoi Régulariser ?	37
3.4.2	Formulation Mathématique	37
3.4.3	Pseudocode : Apprentissage de la Régression Ridge	38
3.4.4	Choix du Paramètre α dans Notre Projet	38
3.5	Surapprentissage, Sous-apprentissage et Compromis Biais-Variance	39
3.5.1	Définitions Fondamentales	39
3.5.2	Le Compromis Biais-Variance	40
3.6	La Régression Logistique (Discussion Théorique)	40
3.6.1	Présentation du Modèle	40
3.6.2	Fonction de Coût : l'Entropie Croisée (Log-Loss)	41
3.6.3	Pourquoi ce Modèle N'est-il Pas Appliqué dans Notre Projet ?	41
3.7	Le Random Forest Regressor	42
3.7.1	Des Arbres de Décision à la Forêt Aléatoire	42
3.7.2	Formulation Mathématique	43
3.7.3	Hyperparamètres Utilisés dans Notre Projet	43
3.8	L'Ensemble Learning : Le Voting Regressor	44
3.8.1	Principe de l'Apprentissage d'Ensemble	44
3.8.2	Formulation Mathématique	44
3.8.3	Justification Théorique : Réduction de la Variance	45
3.9	Synthèse Comparative des Modèles	45

3.10 Conclusion	46
4 Résultats et Analyse	48
4.1 Introduction	48
4.2 Comparaison des Performances Prédicatives (MSE)	48
4.2.1 La Métrique d'Évaluation : Mean Squared Error	49
4.2.2 Résultats Numériques	49
4.3 Analyse Graphique des Prévisions de Volatilité	50
4.3.1 Graphique 1 : Comparaison Globale des Trois Modèles	51
4.3.2 Graphique 2 : Focus sur la Régression Linéaire	51
4.3.3 Graphique 3 : Focus sur la Régression Ridge	52
4.3.4 Graphique 4 : Focus sur le Random Forest	53
4.4 Allocation Dynamique de Portefeuille	53
4.4.1 De la Prévision de Volatilité à l'Allocation d'Actifs	53
4.4.2 Graphique 5 : Évolution du Capital — Régression Linéaire vs Random Forest vs Naïf	54
4.4.3 Graphique 6 : Conclusion — Le Super-Modèle vs Tous	55
4.5 Évaluation Financière : Ratios de Performance	56
4.5.1 Rappel des Formules	56
4.5.2 Résultats du Portefeuille Régression Linéaire	56
4.5.3 Résultats du Super-Portefeuille Ensemble Learning	57
4.5.4 Tableau Comparatif Final	57
4.6 Conclusion	59
Conclusion Générale et Perspectives	61
Bilan du Projet	61
Synthèse des Résultats	61
Limites de l'Étude	62
Perspectives Futures	62
Bibliographie et Webographie	65
Ouvrages et Cours de Référence	65
Articles Scientifiques Fondateurs	65
Documentation des Bibliothèques Python Utilisées	66
Code Source du Projet	66

Table des figures

2.1	Évolution comparative de la volatilité empirique sur 20 jours de TSLA et du SPY (2018-2024)	18
2.2	Architecture globale du pipeline de préparation des données	28
4.1	Comparaison globale des trois modèles face à la volatilité réelle de TSLA sur la période de test	51
4.2	Focus sur les prévisions de la Régression Linéaire Multiple face à la volatilité réelle	52
4.3	Focus sur les prévisions de la Régression Ridge ($\alpha = 10$) — illustration du sous-apprentissage	52
4.4	Focus sur les prévisions du Random Forest Regressor face à la volatilité réelle	53
4.5	Évolution du capital : Portefeuille Régression Linéaire vs Random Forest vs Portefeuille Naïf 50/50 sur la période de test	54
4.6	Comparaison finale : Super-Modèle Ensemble Learning vs Régression Linéaire vs Random Forest vs Stratégie Naïve 50/50	55

Liste des tableaux

2.1	Comparaison des deux actifs du portefeuille	13
2.2	Régimes de marché couverts par la période d'étude (2018–2024)	14
2.3	Aperçu des 10 premières observations des rendements journaliers (R_t). .	16
2.4	Illustration de la prévention du data leakage	23
2.5	Récapitulatif des variables du modèle	24
2.6	Caractéristiques du split Train/Test	26
3.1	Comparaison entre Batch Gradient Descent et Stochastic Gradient Descent	36
3.2	Hyperparamètres du Random Forest Regressor utilisé	44
3.3	Synthèse comparative des modèles étudiés	46
4.1	Comparaison des MSE des quatre modèles sur l'ensemble de test	49
4.2	Ratios financiers du Portefeuille Régression Linéaire	56
4.3	Ratios financiers du Super-Portefeuille Ensemble Learning	57
4.4	Comparaison finale des trois stratégies d'investissement	58

CHAPITRE 1

- ▶ CONTEXTE GÉNÉRAL
- ▶ PROBLÉMATIQUE
- ▶ OBJECTIFS DU PROJET
- ▶ DONNÉES UTILISÉES
- ▶ ORGANISATION DU MÉMOIRE

Chapitre 1

Introduction Générale

1.1 Contexte Général

Au cours des dernières décennies, les marchés financiers mondiaux ont connu des transformations profondes et accélérées. La mondialisation des échanges, l'essor des technologies numériques et la complexité croissante des instruments financiers ont profondément modifié la nature des marchés boursiers. Dans ce contexte, les acteurs financiers qu'il s'agisse de banques d'investissement, de fonds de gestion d'actifs ou d'investisseurs individuels font face à un défi permanent : comprendre, mesurer et anticiper le risque financier.

La **volatilité** est l'un des concepts les plus fondamentaux de la finance moderne. Elle mesure l'amplitude des fluctuations d'un actif financier autour de sa valeur moyenne sur une période donnée. Une volatilité élevée signifie que le prix d'un actif peut varier très fortement en peu de temps, ce qui représente à la fois une opportunité de gain et un risque de perte considérable. À l'inverse, une faible volatilité témoigne d'un actif plus stable et moins risqué. La gestion du risque, et donc la maîtrise de la volatilité, est au cœur de toute stratégie d'investissement professionnelle.

Historiquement, les modèles de mesure du risque reposaient sur des approches statistiques classiques comme le modèle GARCH (*Generalized Autoregressive Conditional Heteroskedasticity*), qui cherche à modéliser la volatilité conditionnelle d'une série temporelle financière. Si ces modèles ont longtemps constitué la référence dans le domaine, ils présentent des limites importantes : ils supposent souvent une distribution gaussienne des rendements, ils peinent à capturer des relations non linéaires complexes, et leur calibration reste délicate en présence de chocs exogènes soudains comme les crises financières.

C'est dans ce contexte que l'**apprentissage automatique** (*Machine Learning*) a progressivement imposé sa présence dans l'univers de la finance quantitative. Grâce à sa capacité à apprendre des patterns complexes à partir de grandes quantités de données, le Machine Learning offre une alternative puissante et flexible aux approches statistiques traditionnelles. Des institutions financières de premier plan comme Goldman Sachs, JPMorgan ou BlackRock ont déjà intégré des algorithmes d'apprentissage automatique dans leurs processus de gestion des risques et d'allocation d'actifs.

Ce projet de fin d'études s'inscrit pleinement dans cette dynamique. Il propose une approche originale combinant des techniques de Machine Learning avec des méthodes d'optimisation financière, dans le but de prévoir la volatilité d'un actif à haute capitalisation (l'action **Tesla (TSLA)**) et d'exploiter ces prévisions pour construire une stratégie d'allocation dynamique de portefeuille plus performante et mieux contrôlée en termes de risque.

1.2 Problématique

La prédiction de la volatilité financière est un problème notoirement difficile. Les marchés boursiers sont des systèmes complexes, influencés par une multitude de facteurs hétérogènes : indicateurs macroéconomiques, décisions de politique monétaire, sentiments des investisseurs, annonces d'entreprises, événements géopolitiques, ou encore phénomènes de contagion entre marchés. Cette complexité rend la modélisation de la volatilité particulièrement ardue.

Les approches traditionnelles de gestion de portefeuille reposent souvent sur des hypothèses simplificatrices. La théorie moderne du portefeuille de Markowitz, par exemple, suppose que les rendements suivent une distribution normale et que les corrélations entre actifs sont stables dans le temps. Ces hypothèses sont rarement vérifiées en pratique, notamment lors de périodes de crise où les corrélations explosent et les distributions de rendements présentent des queues épaisses (*fat tails*).

De plus, les stratégies d'allocation statique — comme la règle naïve du **50%/50%** qui consiste à répartir également son capital entre deux actifs — ignorent complètement l'évolution du risque dans le temps. Un investisseur qui alloue une part fixe de son portefeuille à un actif très volatile comme Tesla ne tient pas compte du fait que le niveau de risque de cet actif peut varier considérablement d'un mois à l'autre, voire d'une semaine à l'autre.

Face à ces limites, plusieurs questions fondamentales se posent naturellement :

- Est-il possible de prévoir la volatilité future d'un actif financier à partir de ses données historiques et de ses indicateurs techniques ?
- Parmi les différents modèles de Machine Learning disponibles, lesquels offrent les meilleures performances pour cette tâche de régression sur données financières ?
- Comment exploiter ces prévisions de volatilité pour construire une stratégie d'allocation dynamique de portefeuille qui adapte en permanence l'exposition au risque ?
- Une telle stratégie pilotée par l'intelligence artificielle peut-elle surpasser une stratégie d'investissement statique simple, mesurée par des ratios financiers reconnus comme le Ratio de Sharpe, le Ratio de Sortino et le Ratio de Calmar ?

Ce sont ces questions qui constituent le fil conducteur de ce projet de fin d'études, et auxquelles nous allons tenter d'apporter des réponses rigoureuses et documentées à travers une démarche scientifique structurée.

1.3 Objectifs du Projet

Ce projet poursuit plusieurs objectifs complémentaires et progressifs, allant de la collecte et la préparation des données jusqu'à l'évaluation financière des stratégies générées par les modèles.

Premier objectif — Prévision de la volatilité par apprentissage automatique :

L'objectif principal est de construire des modèles capables de prévoir la volatilité future de l'action Tesla (**TSLA**) à partir de variables explicatives extraites des données historiques de marché. Cette variable cible est définie comme l'écart-type des rendements journaliers calculé sur une fenêtre glissante de 20 jours de bourse, ce qui correspond approximativement à un mois de trading.

Deuxième objectif — Construction et comparaison de plusieurs modèles :

Afin de déterminer la meilleure approche pour ce problème, plusieurs modèles de Machine Learning sont implémentés, entraînés et comparés :

- La **Régression Linéaire Multiple** (Méthode des Moindres Carrés Ordinaires), comme modèle de référence simple et interprétable.
- La **Régression Ridge** (régularisation L2), pour étudier l'effet de la régularisation sur la prévision de volatilité et illustrer le concept de sous-apprentissage (*underfitting*).
- Le **Random Forest Regressor**, un modèle non linéaire basé sur un ensemble d'arbres de décision, capable de capturer des relations complexes dans les données.

- Un **modèle d'Ensemble Learning** (Voting Regressor) combinant la Régression Linéaire et le Random Forest, pour tirer parti des forces complémentaires de chaque approche.

Troisième objectif — Ingénierie des variables explicatives : Un soin particulier est apporté à la construction des variables d'entrée (*features*) utilisées par les modèles. Ces variables comprennent les rendements des cinq derniers jours de bourse, la volatilité historique récente, ainsi que deux indicateurs techniques reconnus : le **RSI** (*Relative Strength Index*) et le **MACD** (*Moving Average Convergence Divergence*). Une attention particulière est portée à la prévention du *data leakage*, c'est-à-dire à l'élimination de toute fuite d'information du futur vers le passé lors de la construction des variables.

Quatrième objectif — Allocation dynamique de portefeuille : Les prévisions de volatilité générées par les modèles sont ensuite utilisées pour piloter une stratégie d'allocation dynamique de portefeuille basée sur le principe de la **parité des risques** (*Inverse Volatility Weighting*). Cette stratégie ajuste en temps réel les poids alloués à chaque actif (TSLA et SPY) en fonction de leur niveau de risque prédit, dans le but de maintenir une contribution au risque égale entre les deux actifs.

Cinquième objectif — Évaluation des performances financières : Les résultats des différentes stratégies sont évalués et comparés à l'aide de trois ratios financiers standards reconnus par la profession :

- Le **Ratio de Sharpe**, qui mesure le rendement obtenu par unité de risque total.
- Le **Ratio de Sortino**, qui ne pénalise que le risque négatif (les pertes).
- Le **Ratio de Calmar**, qui mesure la résistance du portefeuille aux krachs boursiers en rapportant le rendement annualisé au pire repli historique subi.

1.4 Données Utilisées

Pour mener à bien ce projet, nous avons utilisé les données historiques de cours boursiers de deux actifs soigneusement sélectionnés pour leur complémentarité en termes de profil de risque :

SPY — SPDR S&P 500 ETF Trust : Le SPY est un fonds indiciel coté en bourse (*Exchange-Traded Fund*) qui réplique fidèlement la performance de l'indice boursier américain S&P 500. Cet indice regroupe les 500 plus grandes entreprises cotées aux États-Unis et constitue la référence mondiale pour mesurer la performance du marché actions américain. Le SPY représente dans notre portefeuille l'**actif stable et diversifié**, avec une volatilité historiquement modérée.

TSLA — Tesla Inc. : L'action Tesla est l'un des titres les plus volatils et les plus suivis du marché boursier mondial. Entreprise pionnière dans le domaine des véhicules électriques et des énergies renouvelables, Tesla est connue pour ses fluctuations de cours extrêmement amplifiées, influencées aussi bien par ses résultats financiers que par les déclarations médiatiques de son dirigeant, Elon Musk. TSLA représente dans notre portefeuille l'**actif risqué à haute volatilité**, dont la prévision du risque constitue le cœur de notre problématique.

La période d'étude couvre **six années**, de **janvier 2018 à janvier 2024**. Ce choix est motivé par la nécessité de disposer d'un volume de données suffisant pour entraîner des modèles de Machine Learning de manière robuste, tout en couvrant des périodes de marché très différentes : la montée en puissance de Tesla entre 2019 et 2021, la crise sanitaire liée à la pandémie de COVID-19 en 2020, la période de forte inflation et de hausse des taux d'intérêt en 2022, ainsi que la reprise partielle des marchés en 2023.

Les données ont été téléchargées gratuitement via la bibliothèque Python **yfinance**, qui permet d'accéder à l'ensemble des historiques boursiers disponibles sur Yahoo Finance. Seuls les **prix de clôture ajustés** (*Adjusted Close*) ont été retenus pour les calculs, afin de tenir compte des événements corporatifs tels que les divisions d'actions (*stock splits*) et le versement de dividendes. Le détail complet du traitement et de la transformation de ces données est présenté dans le Chapitre 2.

1.5 Organisation du Mémoire

Ce mémoire est organisé en cinq chapitres distincts, chacun traitant d'un aspect spécifique du projet, depuis la mise en contexte jusqu'à l'analyse des résultats obtenus.

Chapitre 1 — Introduction Générale : Le présent chapitre pose le cadre général du projet. Il présente le contexte financier qui motive notre démarche, formule la problématique centrale, définit les objectifs poursuivis, décrit brièvement les données utilisées et présente l'organisation globale du mémoire.

Chapitre 2 — Préparation et ingénierie des données : Ce chapitre est consacré à la description détaillée des données et à leur transformation en variables exploitables par les modèles de Machine Learning. Il couvre le calcul des rendements journaliers, la construction de la variable cible (la volatilité empirique sur fenêtre glissante de 20 jours), l'ingénierie des variables explicatives (rendements passés, RSI, MACD), la prévention du data leakage par décalage temporel, et enfin la séparation temporelle stricte des données en ensembles d'entraînement et de test.

Chapitre 3 — Modèles et Concepts Mathématiques : Ce chapitre présente en détail les fondements théoriques et mathématiques de chacun des modèles utilisés dans ce projet. La Régression Linéaire Multiple, la Régression Ridge, le Random Forest Regressor et l'Ensemble Learning sont décrits avec leurs équations, leurs fonctions de coût, leurs algorithmes d'optimisation (descente de gradient, descente de gradient stochastique) et leurs pseudocodes. Les concepts clés de surapprentissage (*overfitting*), de sous-apprentissage (*underfitting*) et du compromis biais-variance sont également développés en détail.

Chapitre 4 — Résultats et analyse : Ce chapitre présente et analyse l'ensemble des résultats obtenus. Il compare les performances prédictives des différents modèles à l'aide de l'erreur quadratique moyenne (MSE), illustre les prévisions de volatilité par des graphiques détaillés, et évalue les performances financières des stratégies d'allocation dynamique générées par chaque modèle à travers les Ratios de Sharpe, de Sortino et de Calmar.

Chapitre 5 — Conclusion et Perspectives : Le dernier chapitre synthétise les principaux enseignements du projet, discute de ses limites et ouvre des perspectives d'amélioration et d'extension pour de futurs travaux, notamment l'intégration de modèles de deep learning, l'extension à un portefeuille multi-actifs, ou encore l'utilisation de données alternatives comme le sentiment de marché extrait des réseaux sociaux.

Enfin, une **bibliographie et webographie** rassemble l'ensemble des références scientifiques, ouvrages, articles fondateurs et bibliothèques Python utilisés dans ce projet, ainsi que le lien vers le dépôt GitHub contenant l'intégralité du code source Python développé, permettant de reproduire l'ensemble des résultats présentés dans ce mémoire.

CHAPITRE 2

- ▶ INTRODUCTION
- ▶ ÉTAT DE L'ART
- ▶ PRÉSENTATION DES DONNÉES
- ▶ CALCUL DES RENDEMENTS
- ▶ VARIABLE CIBLE : LA VOLATILITÉ
- ▶ INGÉNIERIE DES VARIABLES
- ▶ SÉPARATION TRAIN / TEST

Chapitre 2

Préparation et Ingénierie des Données

2.1 Introduction

La qualité d'un modèle de Machine Learning repose avant tout sur la qualité des données qui lui sont fournies. Avant d'entraîner le moindre algorithme, il est indispensable de collecter, nettoyer, transformer et structurer les données brutes en un ensemble de variables exploitables. Ce processus, appelé **ingénierie des données** (*data engineering*) et **ingénierie des variables** (*feature engineering*), constitue souvent la partie la plus chronophage et la plus déterminante d'un projet de science des données.

Ce chapitre présente l'ensemble du pipeline de préparation des données mis en œuvre dans ce projet. Il s'articule autour de plusieurs étapes successives : la présentation des sources de données et des actifs sélectionnés, le calcul des rendements journaliers, la construction de la variable cible (la volatilité empirique), l'ingénierie des variables explicatives incluant des indicateurs techniques avancés, la prévention du *data leakage*, et enfin la séparation temporelle stricte des données en ensembles d'entraînement et de test.

Chaque étape est décrite avec ses fondements mathématiques, son pseudocode algorithmique et sa justification méthodologique, afin de garantir la reproductibilité et la rigueur scientifique de l'approche.

2.2 État de l'Art — Approches de Prévision de la Volatilité

Cette section dresse un panorama des méthodes de prévision de la volatilité financière. Nous y examinons les limites des modèles économétriques traditionnels (ARCH/GARCH) face à la complexité des marchés, avant de mettre en évidence les solutions modernes offertes par l'apprentissage automatique, justifiant ainsi le positionnement de notre étude.

2.2.1 Modèles Statistiques Classiques

La modélisation de la volatilité financière a une longue histoire académique. Les premières approches rigoureuses ont émergé dans les années 1980 avec les travaux fondateurs d'Engle (1982) et Bollerslev (1986).

Le modèle ARCH (Autoregressive Conditional Heteroskedasticity) fut proposé par Robert Engle en 1982. L'idée centrale est que la variance conditionnelle d'une série temporelle financière n'est pas constante dans le temps, mais dépend des erreurs passées. Un modèle ARCH(q) s'écrit :

$$\sigma_t^2 = \alpha_0 + \sum_{i=1}^q \alpha_i \varepsilon_{t-i}^2 \quad (2.1)$$

où σ_t^2 est la variance conditionnelle à l'instant t , ε_{t-i} sont les résidus passés, et α_i sont les paramètres du modèle. Ce modèle capture le phénomène bien connu de **clustering de volatilité** : les périodes de forte agitation tendent à se regrouper dans le temps.

Le modèle GARCH (Generalized ARCH), introduit par Bollerslev en 1986, généralise le modèle ARCH en incluant également les variances passées dans l'équation :

$$\sigma_t^2 = \omega + \sum_{i=1}^q \alpha_i \varepsilon_{t-i}^2 + \sum_{j=1}^p \beta_j \sigma_{t-j}^2 \quad (2.2)$$

Le modèle GARCH(1,1), le plus utilisé en pratique, s'écrit simplement :

$$\sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2 + \beta \sigma_{t-1}^2 \quad (2.3)$$

Ces modèles ont longtemps constitué la référence en finance quantitative. Cependant, ils présentent des limites importantes qui motivent l'utilisation du Machine Learning :

- Ils supposent une **distribution gaussienne** des résidus, hypothèse rarement vérifiée en pratique (les rendements financiers présentent des queues épaisses).
- Ils peinent à capturer des **relations non linéaires complexes** entre les variables de marché.
- Leur calibration est délicate en présence de **chocs exogènes soudains** comme les crises financières ou les pandémies.
- Ils n'intègrent pas facilement des **variables externes** comme les indicateurs techniques (RSI, MACD).

2.2.2 Apprentissage Automatique Appliqué à la Finance

Ces dernières années, de nombreux travaux académiques ont exploré l'application des algorithmes de Machine Learning à la prévision de la volatilité financière. Ces approches présentent plusieurs avantages décisifs par rapport aux modèles économétriques classiques.

Les **modèles de régression** (linéaire, Ridge, Lasso) ont été appliqués avec succès à la prévision de la volatilité réalisée à court terme. Ils sont particulièrement appréciés pour leur interprétabilité et leur faible coût computationnel.

Les **méthodes d'ensemble** comme le Random Forest et le Gradient Boosting se sont révélées particulièrement performantes sur des données financières, grâce à leur capacité à capturer des interactions non linéaires complexes entre variables sans nécessiter de spécification a priori du modèle.

2.2.3 Positionnement de Notre Approche

Notre projet se positionne à l'intersection de ces deux mondes. Plutôt que d'utiliser des modèles GARCH, nous adoptons une approche supervisée de Machine Learning, en construisant un ensemble de variables explicatives (*features*) à partir des données historiques de marché. Notre contribution spécifique réside dans :

- La combinaison de **variables de mémoire** (rendements passés) et d'**indicateurs techniques** (RSI, MACD) comme prédicteurs de la volatilité.
- La comparaison systématique de plusieurs modèles de complexité croissante, depuis la régression linéaire jusqu'à l'Ensemble Learning.
- L'exploitation directe des prévisions de volatilité pour une **application financière concrète** : l'allocation dynamique de portefeuille.

2.3 Présentation des Données

Cette section présente les séries temporelles au cœur de notre modélisation, extraites de Yahoo Finance via la bibliothèque `yfinance`. Elle justifie la période d'étude retenue (2018-2024) et détaille les profils de risque complémentaires des deux actifs sélectionnés pour notre portefeuille : l'ETF SPY et l'action Tesla (TSLA).

2.3.1 Source des Données : Yahoo Finance et `yfinance`

Les données utilisées dans ce projet proviennent de **Yahoo Finance**, l'une des plateformes les plus utilisées pour l'accès gratuit aux données boursières historiques. L'accès à ces données est réalisé via la bibliothèque Python `yfinance`, qui fournit une interface simple et efficace pour télécharger les historiques de prix de n'importe quel actif coté en bourse.

La bibliothèque `yfinance` permet de récupérer, pour chaque actif et chaque date, plusieurs types de prix : le prix d'ouverture (*Open*), le prix de clôture (*Close*), le plus haut (*High*), le plus bas (*Low*), le volume échangé (*Volume*), ainsi que le prix de clôture ajusté (*Adjusted Close*).

Dans notre projet, nous utilisons exclusivement le **prix de clôture ajusté** (*Adjusted Close*), noté P_t . Ce prix tient compte des événements corporatifs tels que les divisions d'actions (*stock splits*) et le versement de dividendes, ce qui garantit la continuité et la comparabilité des séries temporelles sur toute la période d'étude.

2.3.2 Actifs Sélectionnés

Deux actifs financiers complémentaires ont été sélectionnés pour constituer notre portefeuille d'étude. Ce choix est motivé par la volonté de confronter deux profils de risque radicalement différents.

Comme le synthétise le tableau 2.1, le choix de ces deux actifs repose sur une asymétrie de risque volontaire : la stabilité historique du SPY sert de valeur refuge face aux fluctuations extrêmes de l'action TSLA, qui constitue la cible principale de nos prévisions de volatilité.

TAB. 2.1 : Comparaison des deux actifs du portefeuille

Caractéristique	SPY	TSLA
Nom complet	SPDR S&P 500 ETF Trust	Tesla, Inc.
Type	ETF (Fonds indiciel coté)	Action individuelle
Marché	NYSE Arca	NASDAQ
Secteur	Diversifié (500 secteurs)	Véhicules électriques / Énergie
Profil de risque	Stable, diversifié	Haute volatilité
Rôle dans le portefeuille	Actif refuge / référence marché	Actif risqué / cible de prévision
Volatilité piquée annuelle	ty- $\approx 15\text{--}20\%$	$\approx 50\text{--}80\%$

SPY — SPDR S&P 500 ETF Trust : Le SPY réplique fidèlement la performance de l'indice S&P 500, qui regroupe les 500 plus grandes entreprises cotées aux États-Unis. Il constitue la référence mondiale pour mesurer la performance du marché actions américain. Dans notre portefeuille, il joue le rôle d'**actif stable et diversifié**, dont la volatilité sert de signal pour l'allocation dynamique.

TSLA — Tesla Inc. : L'action Tesla est l'un des titres les plus volatils et les plus médiatisés du marché boursier mondial. Ses fluctuations extrêmes sont influencées par les résultats financiers trimestriels, les annonces technologiques, et les déclarations publiques de son fondateur Elon Musk. TSLA constitue notre **actif risqué à haute volatilité**, dont la prévision du risque est l'objet central de ce projet.

2.3.3 Période d'Étude et Justification

La période d'étude s'étend sur **six années**, du **1^{er} janvier 2018** au **1^{er} janvier 2024**. Ce choix n'est pas arbitraire : il est motivé par la diversité des régimes de marché couverts. Comme le détaille le tableau 2.2, cette fenêtre temporelle inclut des chocs macroéconomiques majeurs (krach pandémique, poussées inflationnistes), garantissant ainsi que nos modèles seront entraînés sur des cycles de volatilité très variés.

TAB. 2.2 : Régimes de marché couverts par la période d'étude (2018–2024)

Période	Événement	Impact sur les marchés
2018	Guerre commerciale USA–Chine	Volatilité accrue, correction de fin d'année
2019–2020	Montée en puissance de Tesla	TSLA multipliée par 10, volatilité extrême
Mars 2020	Pandémie COVID-19	Krach historique (-34% en un mois), pic de volatilité
2021	Reprise post-COVID	Marchés euphoriques, volatilité en baisse
2022	Inflation et hausse des taux	Correction sévère, TSLA perd 65%
2023	Reprise partielle	Marchés en rebond, volatilité modérée

La diversité de ces régimes de marché est essentielle pour entraîner des modèles robustes, capables de généraliser au-delà des conditions de marché observées pendant l'apprentissage.

2.4 Calcul des Rendements Journaliers

Pour garantir la stationnarité requise par les modèles d'apprentissage automatique, il est indispensable de substituer les prix bruts par leurs variations relatives. Cette section détaille la transformation mathématique des prix de clôture en rendements journaliers arithmétiques, et présente l'algorithme d'extraction automatisée appliqué à nos séries temporelles.

2.4.1 Définition et Formule Mathématique

Les modèles de Machine Learning ne travaillent pas directement sur les prix absolus P_t , mais sur les **rendements journaliers** R_t , qui mesurent la variation relative du prix d'un actif d'un jour à l'autre. Cette transformation présente plusieurs avantages fondamentaux :

- Les rendements sont **stationnaires** (contrairement aux prix qui suivent une tendance), ce qui est une propriété souhaitable pour les modèles statistiques.
- Ils permettent de **comparer** des actifs de niveaux de prix très différents.
- Ils ont une interprétation économique directe : un rendement de +2% signifie un gain de 2% de la valeur investie en une journée.

Le rendement journalier à l'instant t est défini par la formule :

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}} \quad (2.4)$$

où P_t désigne le prix de clôture ajusté à la date t , et P_{t-1} le prix de clôture ajusté à la date précédente $t - 1$. Cette formule donne un rendement exprimé en proportion (par exemple, $R_t = 0.02$ correspond à une hausse de 2%).

En pratique, on utilise parfois le **rendement logarithmique** $r_t = \ln(P_t/P_{t-1})$, qui présente de meilleures propriétés statistiques (additivité temporelle). Pour des rendements journaliers faibles, les deux formules donnent des résultats très proches. Dans ce projet, nous utilisons le rendement arithmétique (Équation 2.4), conformément à l'implémentation Python via la méthode `pct_change()`.

2.4.2 Pseudocode : Téléchargement et Calcul des Rendements

Algorithme 1 : Téléchargement des données et calcul des rendements

Entrée : Liste d'actifs $\mathcal{A} = \{\text{SPY}, \text{TSLA}\}$, date de début d_1 , date de fin d_2

Sortie : DataFrame $\mathbf{R}$ des rendements journaliers

1. Télécharger l'historique des prix via `yfinance.download(\mathcal{A}, d_1, d_2)`
2. Extraire la colonne `Close` $\rightarrow$ matrice de prix $\mathbf{P}$
3. **Pour** chaque actif $a \in \mathcal{A}$:
 - Calculer $R_t^{(a)} = \frac{P_t^{(a)} - P_{t-1}^{(a)}}{P_{t-1}^{(a)}}$ pour tout $t \in [d_1 + 1, d_2]$
4. Supprimer la première ligne (valeur manquante due au décalage)
5. **Retourner** le DataFrame $\mathbf{R}$ de dimensions $(T - 1) \times 2$

L'Algorithme 1 formalise la procédure d'extraction automatisée qui transforme la matrice des prix bruts en rendements journaliers. Cette étape inclut la suppression indispensable de la première observation (valeur manquante) inhérente au calcul par décalage temporel, garantissant ainsi l'intégrité de la série de données.

TAB. 2.3 : Aperçu des 10 premières observations des rendements journaliers (R_t).

Date	SPY	TSLA
2018-01-03	0.006325	-0.010233
2018-01-04	0.004215	-0.008290
2018-01-05	0.006664	0.006230
2018-01-08	0.001828	0.062638
2018-01-09	0.002263	-0.008085
2018-01-10	-0.001530	0.003326
2018-01-11	0.007296	0.009409
2018-01-12	0.006519	-0.005119
2018-01-16	-0.003418	0.011421
2018-01-17	0.009532	0.020879

Le tableau 2.3 présente un extrait des premiers rendements journaliers calculés suite à la transformation des prix de clôture. Dès ces premières observations temporelles, l'écart de comportement est manifeste : les variations du SPY demeurent modérées, tandis que l'action TSLA affiche des fluctuations d'une amplitude nettement supérieure (notamment le 8 janvier), illustrant empiriquement sa plus forte volatilité.

2.5 Construction de la Variable Cible : La Volatilité Empirique

Dans tout modèle d'apprentissage supervisé, la définition rigoureuse de la variable à prédire est une étape déterminante. Cette section détaille la formulation mathématique de la volatilité empirique, évaluée à travers l'écart-type des rendements sur une fenêtre glissante de 20 jours. Cet indicateur de risque constituera la cible fondamentale que nos algorithmes devront apprendre à anticiper.

2.5.1 Définition Mathématique

La **variable cible** est la grandeur que nos modèles de Machine Learning doivent apprendre à prédire. Dans ce projet, il s'agit de la **volatilité empirique**, définie comme l'écart-type des rendements journaliers calculé sur une fenêtre glissante de $N = 20$ jours de bourse.

Formellement, la volatilité empirique à l'instant t est définie par :

$$\sigma_t = \sqrt{\frac{1}{N-1} \sum_{i=0}^{N-1} (R_{t-i} - \bar{R})^2} \quad (2.5)$$

où :

- $N = 20$ est la taille de la fenêtre glissante (environ 1 mois de trading),
- R_{t-i} est le rendement journalier à l'instant $t - i$,
- $\bar{R} = \frac{1}{N} \sum_{i=0}^{N-1} R_{t-i}$ est la moyenne des rendements sur la fenêtre de 20 jours.

Cette mesure est calculée pour chaque jour t en faisant glisser la fenêtre de 20 jours sur toute la série temporelle. On obtient ainsi une **série temporelle de volatilité** $(\sigma_t)_t$, qui capture l'évolution du niveau de risque de l'actif au fil du temps.

2.5.2 Justification du Choix de la Fenêtre de 20 Jours

Le choix d'une fenêtre de $N = 20$ jours repose sur plusieurs arguments :

- **Correspondance avec le mois de trading** : Un mois de bourse comprend environ 20 jours ouvrables. La volatilité sur 20 jours est donc un indicateur du risque mensuel, une échelle pertinente pour la gestion de portefeuille.
- **Standard de l'industrie** : La fenêtre de 20 jours est largement utilisée dans la littérature financière et dans les pratiques professionnelles de gestion des risques.
- **Compromis biais-variance** : Une fenêtre trop courte (5 jours) serait trop bruitée, tandis qu'une fenêtre trop longue (60 jours) laisserait passer les changements rapides de régime de volatilité.

2.5.3 Pseudocode : Calcul de la Volatilité Glissante

L'Algorithme 2 formalise le processus de calcul de la volatilité empirique par fenêtre glissante. En appliquant une fenêtre mobile de 20 jours sur la série des rendements, l'algorithme évalue rigoureusement l'écart-type à chaque instant t . L'exclusion finale des premières observations assure que chaque valeur cible est calculée sur une période

complète, garantissant ainsi la viabilité des données pour la phase d'apprentissage.

Algorithme 2 : Calcul de la volatilité empirique glissante

Entrée : DataFrame $\mathbf{R}$ des rendements, fenêtre $N = 20$

Sortie : Série temporelle de volatilité σ

1. **Pour** chaque date t allant du jour N jusqu'à T :
 - Extraire la sous-série $\{R_{t-N+1}, \dots, R_t\}$
 - Calculer la moyenne $\bar{R} = \frac{1}{N} \sum_{i=0}^{N-1} R_{t-i}$
 - Calculer l'écart-type $\sigma_t = \sqrt{\frac{1}{N-1} \sum_{i=0}^{N-1} (R_{t-i} - \bar{R})^2}$
2. Supprimer les $N - 1$ premières lignes (fenêtre incomplète)
3. **Retourner** la série $\sigma = (\sigma_t)_{t=N}^T$

La figure 2.1 illustre la dynamique temporelle de la volatilité empirique glissante calculée pour les deux actifs étudiés. Elle met en évidence l'amplitude extrême des fluctuations de l'action TSLA, notamment lors des chocs de marché de 2020 et 2022, contrastant fortement avec la trajectoire plus stable de l'ETF SPY. Cette représentation visuelle confirme empiriquement l'asymétrie de risque indispensable à l'entraînement de nos modèles d'apprentissage.

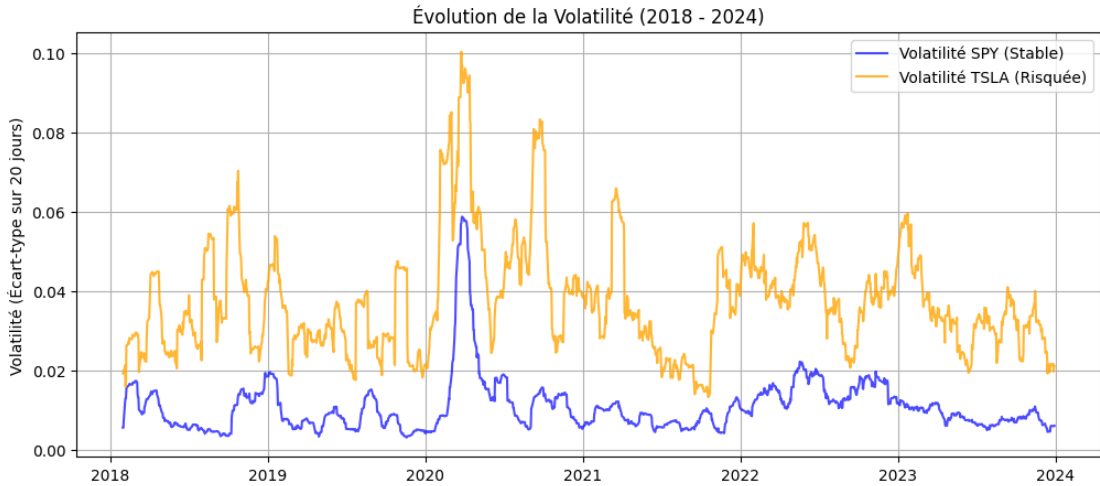


FIG. 2.1 : Évolution comparative de la volatilité empirique sur 20 jours de TSLA et du SPY (2018-2024).

2.6 Ingénierie des Variables Explicatives (Feature Engineering)

L'ingénierie des variables (*feature engineering*) est l'étape qui consiste à construire, à partir des données brutes, un ensemble de variables explicatives (*features*) pertinentes pour les modèles de Machine Learning. C'est l'étape la plus créative et la plus déterminante d'un projet de science des données.

Dans ce projet, sept variables explicatives ont été construites, regroupées en trois catégories : les variables de mémoire courte, la volatilité historique, et les indicateurs techniques avancés.

2.6.1 Variables de Mémoire Courte : Les Rendements Passés

Les marchés financiers présentent une certaine **autocorrélation à court terme** : les rendements passés contiennent une information partielle sur le risque futur. Nous incluons donc les rendements journaliers des cinq derniers jours de bourse comme variables explicatives :

$$\text{Rendement}_{J-k} = R_{t-k}, \quad k = 1, 2, 3, 4, 5 \quad (2.6)$$

Ces cinq variables de décalage (*lag variables*) donnent au modèle une "mémoire courte" des mouvements récents du marché. Un jour de forte baisse (rendement très négatif) est souvent suivi d'une période de volatilité accrue.

2.6.2 La Volatilité Historique Décalée

En plus des rendements passés, nous incluons la **volatilité historique décalée de 5 jours** :

$$\text{Volatilité}_{J-5} = \sigma_{t-5} \quad (2.7)$$

Cette variable capture la persistance de la volatilité : une période de forte volatilité tend à en engendrer une autre (phénomène de *volatility clustering*, bien documenté dans la littérature financière depuis les travaux de Mandelbrot).

2.6.3 Le RSI (Relative Strength Index)

Définition et Intuition

Le **RSI (Relative Strength Index)**, développé par J. Welles Wilder en 1978, est un indicateur de dynamique (*momentum indicator*) qui mesure la vitesse et l'amplitude des variations de prix récentes. Il est exprimé sur une échelle de 0 à 100.

- Un RSI **supérieur à 70** indique qu'un actif est en situation de **surachat** (*overbought*) : le prix a monté trop vite et une correction est probable.
- Un RSI **inférieur à 30** indique une situation de **survente** (*oversold*) : le prix a chuté excessivement et un rebond est envisageable.
- Ces situations extrêmes sont souvent associées à une **volatilité accrue**, ce qui justifie son inclusion dans notre modèle.

Formule Mathématique Complète

Le calcul du RSI se déroule en plusieurs étapes :

Étape 1 : Calculer la variation de prix journalière :

$$\Delta_t = P_t - P_{t-1} \quad (2.8)$$

Étape 2 : Séparer les gains et les pertes :

$$G_t = \max(\Delta_t, 0), \quad L_t = \max(-\Delta_t, 0) \quad (2.9)$$

Étape 3 : Calculer les moyennes mobiles exponentielles des gains et des pertes sur 14 jours (méthode de Wilder, équivalente à une EMA avec $\alpha = 1/14$) :

$$\overline{G}_t = \text{MME}_{14}(G_t), \quad \overline{L}_t = \text{MME}_{14}(L_t) \quad (2.10)$$

Étape 4 : Calculer le rapport de force (*Relative Strength*) :

$$RS_t = \frac{\overline{G}_t}{\overline{L}_t} \quad (2.11)$$

Étape 5 : Normaliser sur l'échelle 0-100 :

$$RSI_t = 100 - \frac{100}{1 + RS_t} \quad (2.12)$$

Pseudocode : Calcul du RSI

L'Algorithme 3 détaille les étapes de calcul du *Relative Strength Index* (RSI) sur une période standard de 14 jours. Il isole dans un premier temps les gains et les pertes journaliers avant de leur appliquer un lissage exponentiel spécifique basé sur la méthode de Wilder. La normalisation mathématique finale génère un oscillateur strictement borné entre 0 et 100, fournissant ainsi à nos modèles une mesure robuste de la dynamique directionnelle des prix.

Algorithme 3 : Calcul du RSI sur 14 jours

Entrée : Série de prix $\mathbf{P}$, période $n = 14$

Sortie : Série **RSI**

1. Calculer les variations journalières : $\Delta_t = P_t - P_{t-1}$
2. Isoler les gains : $G_t = \max(\Delta_t, 0)$
3. Isoler les pertes : $L_t = \max(-\Delta_t, 0)$
4. Calculer la MME des gains sur n jours : $\bar{G}_t = \text{MME}_n(G_t)$
— avec paramètre de lissage $\alpha = \frac{1}{n-1}$ (méthode Wilder)
5. Calculer la MME des pertes sur n jours : $\bar{L}_t = \text{MME}_n(L_t)$
6. Calculer $RS_t = \bar{G}_t / \bar{L}_t$
7. Calculer $RSI_t = 100 - \frac{100}{1 + RS_t}$
8. **Retourner** la série **RSI**

2.6.4 Le MACD (Moving Average Convergence Divergence)

Définition et Intuition

Le **MACD** est un indicateur de tendance (*trend-following indicator*) introduit par Gerald Appel dans les années 1970. Il mesure la différence entre deux moyennes mobiles exponentielles de périodes différentes, capturant ainsi le momentum à court terme par rapport à la tendance à moyen terme.

L'idée centrale est la suivante : lorsque la MME à court terme (12 jours) dépasse la MME à long terme (26 jours), cela signifie que le prix accélère à la hausse, ce qui peut être associé à une augmentation de la volatilité.

Formule Mathématique

La Moyenne Mobile Exponentielle (**MME**) avec une période n est définie par la formule récursive suivante :

$$\text{MME}_t^{(n)} = \alpha \cdot P_t + (1 - \alpha) \cdot \text{MME}_{t-1}^{(n)}, \quad \text{avec } \alpha = \frac{2}{n+1} \quad (2.13)$$

Cette formule attribue un poids plus important aux prix récents (via le facteur α) et un poids décroissant aux prix plus anciens (via $(1 - \alpha)$). Plus n est grand, plus la MME est "lente" et réactive aux tendances long terme.

Le MACD est alors calculé comme la différence entre la MME à 12 jours et la MME à 26 jours :

$$\text{MACD}_t = \text{MME}_{12}(P_t) - \text{MME}_{26}(P_t) \quad (2.14)$$

- $\text{MACD}_t > 0$: la tendance à court terme est haussière (prix monte plus vite que la tendance long terme).
- $\text{MACD}_t < 0$: la tendance à court terme est baissière.
- Un **croisement du MACD à zéro** signale souvent un changement de tendance, fréquemment accompagné d'une hausse de la volatilité.

Pseudocode : Calcul du MACD

Algorithme 4 : Calcul du MACD

Entrée : Série de prix **P**, périodes $n_1 = 12$, $n_2 = 26$

Sortie : Série **MACD**

1. Calculer $\alpha_1 = \frac{2}{n_1+1} = \frac{2}{13}$ (facteur EMA 12 jours)
2. Calculer $\alpha_2 = \frac{2}{n_2+1} = \frac{2}{27}$ (facteur EMA 26 jours)
3. Calculer MME_{12} : pour $t = 1$ à T :
— $\text{MME}_{12,t} = \alpha_1 \cdot P_t + (1 - \alpha_1) \cdot \text{MME}_{12,t-1}$
4. Calculer MME_{26} : pour $t = 1$ à T :
— $\text{MME}_{26,t} = \alpha_2 \cdot P_t + (1 - \alpha_2) \cdot \text{MME}_{26,t-1}$
5. Calculer $\text{MACD}_t = \text{MME}_{12,t} - \text{MME}_{26,t}$ pour tout t
6. **Retourner** la série **MACD**

L'Algorithme 4 explicite la construction de l'indicateur MACD (*Moving Average Convergence Divergence*). En formalisant la différence entre une moyenne mobile exponentielle rapide calculée sur 12 jours et une moyenne lente sur 26 jours, cette procédure mathématique capture l'élan directionnel (*momentum*) des actifs. L'intégration de cette métrique enrichit l'espace des caractéristiques, permettant aux modèles d'apprentissage de mieux anticiper les retournements de tendance.

2.6.5 Prévention du Data Leakage

Le **data leakage** (ou fuite de données) est l'une des erreurs les plus graves et les plus sournoises en Machine Learning. Il survient lorsque le modèle, lors de son entraînement, a accès à des informations qui ne seraient pas disponibles en conditions réelles de prédiction.

Dans notre contexte, le problème est le suivant : si nous calculons le RSI à la date t et que nous l'utilisons directement pour prédire la volatilité à la date t , nous créons une fuite d'information. En effet, en conditions réelles, un investisseur qui veut connaître la volatilité de demain (t) ne dispose que des indicateurs d'aujourd'hui (t) ou d'avant ($t - 1, t - 2, \dots$).

La règle fondamentale est : à la date t , le modèle ne peut utiliser que des informations connues **avant** la date t . Toutes les variables explicatives sont donc décalées d'un jour en arrière via l'opération `.shift(1)` :

$$\text{Feature}_t^{\text{corrigée}} = \text{Feature}_{t-1}^{\text{brute}} \quad (2.15)$$

Le tableau 2.4 illustre le problème de fuite de données et sa solution technique :

TAB. 2.4 : Illustration de la prévention du data leakage

Date	Sans correction (leakage)	Avec correction (.shift(1))
Lundi	RSI du Lundi prédit volatilité du Lundi	RSI du Vendredi prédit volatilité du Lundi
Mardi	RSI du Mardi prédit volatilité du Mardi	RSI du Lundi prédit volatilité du Mardi
Mercredi	RSI du Mercredi prédit volatilité du Mercredi	RSI du Mardi prédit volatilité du Mercredi

2.7 Construction du Jeu de Données Final

Cette section synthétise l'assemblage de la matrice de données définitive destinée à l'apprentissage automatique. En fusionnant la volatilité cible avec les indicateurs techniques rigoureusement décalés, puis en purgeant les valeurs manquantes inhérentes aux fenêtres glissantes, nous consolidons une base d'entraînement structurée, robuste et exempte de tout biais d'anticipation.

2.7.1 Récapitulatif des Variables

L'ensemble des variables explicatives construites constitue notre **matrice de features $\mathbf{X}$** , que nous assemblons avec la variable cible $\mathbf{y}$ pour former le jeu de données complet $\mathcal{D}$.

Comme le synthétise le tableau 2.5, le modèle s'appuie sur un ensemble de **8 variables explicatives** pour anticiper la volatilité future. Le jeu de données final est obtenu par concaténation de ces caractéristiques (features) et de la variable cible. Cette opération est immédiatement suivie d'une purge rigoureuse des observations incomplètes via la fonction `dropna()`, garantissant ainsi une matrice d'entraînement saine et exempte de données manquantes.

TAB. 2.5 : Récapitulatif des variables du modèle

Variable	Type	Description
R_{t-1} à R_{t-5}	Feature ($\times 5$)	Rendements journaliers TSLA des 5 derniers jours
σ_{t-5}	Feature ($\times 1$)	Volatilité historique TSLA décalée de 5 jours
RSI_{t-1}	Feature ($\times 1$)	RSI 14 jours décalé d'un jour
$MACD_{t-1}$	Feature ($\times 1$)	MACD décalé d'un jour
σ_t	Cible ($\mathbf{y}$)	Volatilité empirique TSLA sur 20 jours à prédire

2.7.2 Pseudocode : Construction du Jeu de Données Final

L'Algorithme 5 détaille la procédure de concaténation des variables explicatives et de la variable cible pour finaliser la matrice d'apprentissage.

Algorithme 5 : Construction du jeu de données final

Entrée : Rendements $\mathbf{R}$, Volatilité σ , RSI $\mathbf{RSI}$, MACD $\mathbf{MACD}$

Sortie : Matrices $\mathbf{X}$ (features) et vecteur $\mathbf{y}$ (cible)

1. Créer un DataFrame vide **features**
2. **Pour** $k = 1$ à 5 :
 - Ajouter la colonne $R_{t-k} = \text{TSLA_rendements.shift}(k)$
3. Ajouter la colonne $\sigma_{t-5} = \text{volatilite_TSLA.shift}(5)$
4. Ajouter la colonne $RSI_{t-1} = \text{RSI.shift}(1)$
5. Ajouter la colonne $MACD_{t-1} = \text{MACD.shift}(1)$
6. Concaténer **features** avec la cible σ_t (TSLA)
7. Supprimer toutes les lignes contenant des valeurs manquantes (**dropna**)
8. Séparer : $\mathbf{X}$ = toutes les colonnes sauf la cible, $\mathbf{y}$ = colonne cible
9. **Retourner** $(\mathbf{X}, \mathbf{y})$

2.8 Séparation Temporelle Train / Test

Afin de valider la capacité de généralisation de nos modèles face à des données inédites, nous appliquons une séparation temporelle stricte de 80% pour l'entraînement et 20% pour le test. Cette approche préserve l'ordre chronologique des observations, évitant ainsi toute contamination informationnelle entre les périodes et permettant une évaluation réaliste de la performance financière finale, ou *Exit Value*.

2.8.1 Pourquoi Pas de Split Aléatoire en Finance ?

En Machine Learning classique, il est courant de diviser aléatoirement le jeu de données en ensemble d'entraînement et ensemble de test. Cependant, cette pratique est **formellement interdite** lorsqu'on travaille avec des données temporelles financières, pour la raison suivante :

Si l'on sélectionne aléatoirement des observations, le modèle peut être entraîné sur des données de 2023 et testé sur des données de 2019. Or, en 2023, les modèles auraient "vu" des informations futures par rapport à 2019. Cela constitue un **look-ahead bias** (biais de regard vers l'avenir) qui gonfle artificiellement les performances apparentes du modèle.

La règle fondamentale en finance est : **le passé entraîne, le futur teste**. Autrement dit, le modèle apprend uniquement à partir de données historiques, et est évalué sur des données futures qu'il n'a jamais vues.

2.8.2 Le Split Temporel Strict 80% / 20%

Nous appliquons une **séparation temporelle stricte** : les 80 premiers pourcents des observations chronologiques constituent l'ensemble d'entraînement (*Train set*), et les 20% restants (les plus récents) constituent l'ensemble de test (*Test set*).

Le tableau 2.6 détaille la configuration du découpage temporel effectué sur notre jeu de données.

TAB. 2.6 : Caractéristiques du split Train/Test

Paramètre	Ensemble Train		Ensemble Test
Proportion	80%		20%
Période approximative	Jan 2018 — Déc 2022		Jan 2023 — Jan 2024
Rôle	Apprentissage du modèle		Évaluation des performances
Caractéristique	Données connues	passées	Données futures inconnues

Cette segmentation temporelle garantit que le modèle est exclusivement entraîné sur des séquences historiques avant d'être confronté à la période de test, simulant ainsi un environnement de prédiction réelle.

À l'issue de cette opération, notre jeu de données complet a été divisé comme suit :

- **Ensemble d'entraînement (*Train set*)** : 1187 jours de cotation.
- **Ensemble de test (*Test set*)** : 297 jours de cotation.

2.8.3 Pseudocode : Split Temporel

L'Algorithme 6 formalise le découpage chronologique des données en conservant l'intégrité des séquences temporelles pour l'entraînement et le test.

Algorithme 6 : Séparation temporelle Train / Test

Entrée : Matrices $\mathbf{X}$, $\mathbf{y}$, proportion d'entraînement $p = 0.80$

Sortie : $\mathbf{X}_{\text{train}}$, $\mathbf{y}_{\text{train}}$, $\mathbf{X}_{\text{test}}$, $\mathbf{y}_{\text{test}}$

1. Calculer l'indice de coupure : $i^* = \lfloor p \times |\mathbf{X}| \rfloor$
2. $\mathbf{X}_{\text{train}} \leftarrow \mathbf{X}[0 : i^*]$ (80% des premières observations, chronologiquement)
3. $\mathbf{y}_{\text{train}} \leftarrow \mathbf{y}[0 : i^*]$
4. $\mathbf{X}_{\text{test}} \leftarrow \mathbf{X}[i^* : |\mathbf{X}|]$ (20% des dernières observations)
5. $\mathbf{y}_{\text{test}} \leftarrow \mathbf{y}[i^* : |\mathbf{y}|]$
6. **Retourner** ($\mathbf{X}_{\text{train}}$, $\mathbf{y}_{\text{train}}$, $\mathbf{X}_{\text{test}}$, $\mathbf{y}_{\text{test}}$)

2.9 Synthèse : Pipeline Complet de Préparation des Données

L'ensemble des étapes décrites dans ce chapitre forme un **pipeline séquentiel** de préparation des données. Chaque étape transforme les données brutes en une représentation de plus en plus riche et exploitable par les modèles de Machine Learning. Le schéma suivant résume ce pipeline de bout en bout :

La rigueur de cette chaîne de traitement, de la récupération des prix historiques jusqu'au découpage temporel, constitue le socle indispensable à la fiabilité de nos prédictions. En automatisant la création des indicateurs techniques et en garantissant l'absence de fuite de données, nous assurons que le modèle apprend des relations causales authentiques. Cette structuration méthodique permet à la fin d'isoler la performance intrinsèque de nos algorithmes de celle induite par les biais de construction. Le pipeline ainsi constitué offre une base robuste pour l'étape suivante, dédiée à l'entraînement et à l'optimisation des modèles de régression.

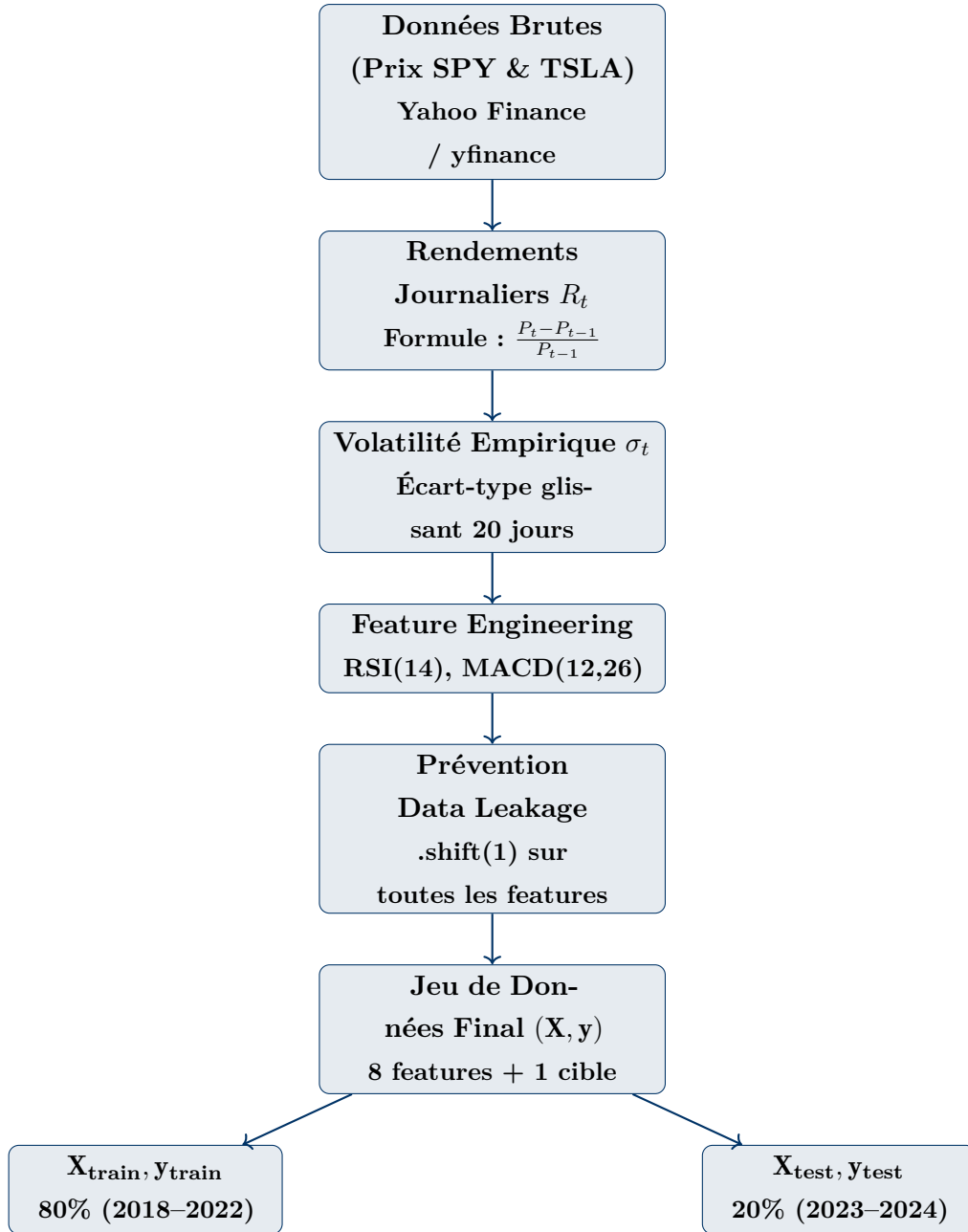


FIG. 2.2 : Architecture globale du pipeline de préparation des données

Ce pipeline, illustré par la figure 2.2, garantit que chaque observation du jeu de données final respecte le principe de causalité temporelle : **les features de la ligne t ne contiennent que des informations disponibles avant la date t** , et les ensembles Train et Test sont strictement séparés dans le temps.

2.10 Conclusion

Ce chapitre a présenté de manière exhaustive l'ensemble du processus de préparation et d'ingénierie des données mis en œuvre dans ce projet. À partir de données brutes de

prix boursiers téléchargées via **yfinance**, nous avons construit un jeu de données structuré comprenant 8 variables explicatives soigneusement choisies pour leur pertinence financière.

Les points clés de ce chapitre sont les suivants :

- La **variable cible** est la volatilité empirique sur fenêtre glissante de 20 jours, un indicateur standard du risque financier à court terme.
- Les **variables explicatives** combinent des variables de mémoire (rendements passés, volatilité historique) et des indicateurs techniques avancés (RSI, MACD).
- La **prévention du data leakage** via le décalage temporel est une étape indispensable pour garantir l'intégrité du processus d'apprentissage.
- La **séparation temporelle stricte** 80%/20% assure que les performances mesurées sur l'ensemble de test sont représentatives des performances réelles en conditions de marché.

Le chapitre suivant présente les fondements mathématiques et algorithmiques des modèles de Machine Learning qui seront entraînés sur ce jeu de données.

CHAPITRE 3

- ▶ INTRODUCTION
- ▶ LA RÉGRESSION LINÉAIRE MULTIPLE
- ▶ LE COMPROMIS BIAIS-VARIANCE
- ▶ LA RÉGRESSION RIDGE (PÉNALITÉ L2)
- ▶ LE RANDOM FOREST REGRESSOR
- ▶ ENSEMBLE LEARNING (VOTING REGRESSOR)
- ▶ ÉVALUATION ET MÉTRIQUES D'ERREUR

Chapitre 3

Méthodologie : Modèles et Concepts Mathématiques

3.1 Introduction

Après avoir construit, dans le chapitre précédent, un jeu de données structuré composé de huit variables explicatives et d'une variable cible (la volatilité empirique), il convient à présent de présenter en détail les modèles d'apprentissage automatique mobilisés pour résoudre ce problème de régression. Ce chapitre constitue le cœur méthodologique de ce mémoire.

Afin de garantir une démarche scientifique rigoureuse, une attention particulière est portée à la formulation mathématique de chaque algorithme. Chaque section détaille le cadre théorique du modèle, sa fonction de coût, son mécanisme d'optimisation et son pseudocode correspondant. Une discussion approfondie des concepts fondamentaux du Machine Learning — le **surapprentissage** (*overfitting*), le **sous-apprentissage** (*underfitting*) et le **compromis biais-variance** — est également proposée, ces notions étant essentielles pour interpréter avec justesse les résultats présentés au Chapitre 4.

Quatre modèles de régression sont étudiés en détail : la **Régression Linéaire Multiple**, la **Régression Ridge**, le **Random Forest Regressor** et le modèle d'**Ensemble Learning** (Voting Regressor) qui les combine. À des fins de complétude théorique et de comparaison analytique, la **Régression Logistique** est également présentée à titre de discussion, bien qu'elle ne soit pas directement appliquée à notre problématique de régression.

Enfin, pour chaque modèle linéaire, deux algorithmes d'optimisation classiques sont étudiés : la **descente de gradient** (*Gradient Descent*) et sa variante **stochastique**

(*Stochastic Gradient Descent*), leur compréhension étant fondamentale pour appréhender la dynamique d'apprentissage des algorithmes modernes.

3.2 La Régression Linéaire Multiple

La régression linéaire multiple sert de modèle de référence pour établir une relation entre les variables explicatives et la volatilité cible. Cette méthode cherche à déterminer le poids optimal de chaque indicateur technique pour anticiper le niveau futur de volatilité.

En supposant que la volatilité suit une tendance globalement linéaire par rapport à ces indicateurs, le modèle estime automatiquement les coefficients qui minimisent l'écart entre les prédictions et les valeurs réelles observées durant la phase d'entraînement. C'est une approche efficace pour identifier quelles variables ont une influence significative sur les variations de volatilité de l'actif.

3.2.1 Formulation du Modèle

La **Régression Linéaire Multiple**, également appelée Méthode des Moindres Carrés Ordinaires (MCO, ou *Ordinary Least Squares*, OLS), constitue le modèle de référence le plus simple et le plus interprétable de notre étude. Elle suppose une relation **linéaire** entre les variables explicatives $x_1, x_2, \dots, x_n$ et la variable cible y (la volatilité future de TSLA).

Pour une observation j , le modèle s'écrit :

$$\hat{y}_j = w_0 + \sum_{i=1}^n w_i x_{i,j} \quad (3.1)$$

où :

- $\hat{y}_j$ est la volatilité prédite pour l'observation j ,
- w_0 est le terme constant (*biais* ou *intercept*),
- w_i sont les coefficients (poids) associés à chacune des $n = 8$ variables explicatives $x_{i,j}$ (les rendements passés, la volatilité historique, le RSI et le MACD).

Sous forme matricielle, en notant $\mathbf{w} = (w_0, w_1, \dots, w_n)^\top$ le vecteur de poids et $\mathbf{x}_j = (1, x_{1,j}, \dots, x_{n,j})^\top$ le vecteur des features augmenté d'une constante, le modèle s'écrit simplement :

$$\hat{y}_j = \mathbf{w}^\top \mathbf{x}_j \quad (3.2)$$

3.2.2 La Fonction de Coût : l'Erreur Quadratique Moyenne

L'objectif de l'apprentissage est de trouver le vecteur de poids $\mathbf{w}$ qui minimise l'écart entre les prédictions $\hat{y}_j$ du modèle et les valeurs réelles y_j observées dans les données d'entraînement. Cet écart est mesuré par la **fonction de coût** (*loss function*), également appelée fonction objective.

Pour la Régression Linéaire, la fonction de coût standard est l'**Erreur Quadratique Moyenne** (MSE, *Mean Squared Error*) :

$$J(\mathbf{w}) = \frac{1}{m} \sum_{j=1}^m (y_j - \hat{y}_j)^2 = \frac{1}{m} \sum_{j=1}^m (y_j - \mathbf{w}^\top \mathbf{x}_j)^2 \quad (3.3)$$

où m est le nombre d'observations dans l'ensemble d'entraînement. Le carré de l'erreur est utilisé pour deux raisons fondamentales : il rend la fonction **différentiable** en tout point (ce qui permet l'optimisation par calcul différentiel), et il **pénalise davantage les grandes erreurs** que les petites, ce qui est souhaitable pour éviter les prédictions très éloignées de la réalité.

L'apprentissage du modèle consiste donc à résoudre le problème d'optimisation suivant :

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} J(\mathbf{w}) \quad (3.4)$$

Dans le cas de la Régression Linéaire simple, ce problème admet une solution analytique exacte (la solution des équations normales) : $\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$. Cependant, lorsque le nombre de variables ou d'observations devient important, cette solution analytique devient coûteuse à calculer (en raison de l'inversion matricielle), et l'on préfère recourir à des méthodes d'optimisation itératives telles que la **descente de gradient**, présentée dans la section suivante.

3.3 Algorithmes d'Optimisation : Descente de Gradient

La descente de gradient est l'algorithme utilisé pour entraîner le modèle de régression en ajustant ses coefficients pour minimiser l'erreur de prédiction. Par une approche itérative, l'algorithme progresse vers les valeurs optimales en suivant la pente descendante de la fonction de coût. Ce processus permet de converger progressivement vers la configuration paramétrique offrant la meilleure précision.

3.3.1 Principe de la Descente de Gradient

La **descente de gradient** (*Gradient Descent*, GD) est un algorithme d'optimisation itératif qui permet de trouver le minimum d'une fonction de coût $J(\mathbf{w})$ sans avoir besoin de calculer une solution analytique exacte. L'idée intuitive est la suivante : à chaque itération, on ajuste légèrement les poids $\mathbf{w}$ dans la direction qui fait **diminuer le plus rapidement** la fonction de coût, c'est-à-dire dans la direction opposée au **gradient** de J .

Le gradient de la fonction de coût MSE (Equation 3.3) par rapport au poids w_i s'écrit :

$$\frac{\partial J(\mathbf{w})}{\partial w_i} = -\frac{2}{m} \sum_{j=1}^m (y_j - \hat{y}_j) x_{i,j} \quad (3.5)$$

La règle de mise à jour des poids à chaque itération t de l'algorithme est alors donnée par :

$$w_i^{(t+1)} = w_i^{(t)} - \eta \frac{\partial J(\mathbf{w}^{(t)})}{\partial w_i} \quad (3.6)$$

où η (*eta*) est le **taux d'apprentissage** (*learning rate*), un hyperparamètre crucial qui contrôle la taille du pas effectué à chaque itération. Un taux d'apprentissage trop élevé peut faire diverger l'algorithme (oscillations autour du minimum sans jamais converger), tandis qu'un taux trop faible ralentit considérablement la convergence.

On parle de *descente de gradient par batch* (*Batch Gradient Descent*) lorsque le gradient de l'Equation 3.5 est calculé sur l'**intégralité** des m observations d'entraînement à chaque itération. Cette approche garantit une convergence stable vers le minimum, mais devient très coûteuse en temps de calcul lorsque m est grand.

3.3.2 Pseudocode : Descente de Gradient (Batch)

L'Algorithme 7 détaille la procédure de descente de gradient par lots qui ajuste les paramètres du modèle sur l'ensemble complet des données. À chaque itération, le gradient global est calculé afin de mettre à jour simultanément tous les poids dans la direction qui réduit l'erreur. Ce processus itératif permet de converger progressivement vers les coefficients optimaux qui minimisent la fonction de coût. Grâce au taux d'apprentissage, l'algorithme assure une progression stable et contrôlée vers la solution finale.

Algorithme 7 : Descente de Gradient (Batch Gradient Descent)**Entrée :** Données $(\mathbf{X}, \mathbf{y})$, taux d'apprentissage η , nombre d'itérations T **Sortie :** Vecteur de poids optimal $\mathbf{w}^*$

1. Initialiser $\mathbf{w}^{(0)}$ aléatoirement (ou à zéro)
2. **Pour** $t = 0$ à $T - 1$:
 - Calculer les prédictions : $\hat{y}_j = \mathbf{w}^{(t)\top} \mathbf{x}_j$ pour tout $j = 1, \dots, m$
 - Calculer le gradient global : $\nabla J(\mathbf{w}^{(t)}) = -\frac{2}{m} \sum_{j=1}^m (y_j - \hat{y}_j) \mathbf{x}_j$
 - Mettre à jour les poids : $\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \eta \nabla J(\mathbf{w}^{(t)})$
3. **Retourner** $\mathbf{w}^* = \mathbf{w}^{(T)}$

3.3.3 La Descente de Gradient Stochastique (SGD)

La **descente de gradient stochastique** (*Stochastic Gradient Descent*, SGD) est une variante de l'algorithme précédent, particulièrement utile lorsque le volume de données devient important. Plutôt que de calculer le gradient sur l'intégralité des m observations à chaque itération, le SGD met à jour les poids en utilisant **une seule observation tirée aléatoirement** (ou un petit *mini-batch*) à la fois.

La règle de mise à jour pour une observation j tirée aléatoirement devient :

$$w_i^{(t+1)} = w_i^{(t)} + 2\eta (y_j - \hat{y}_j) x_{i,j} \quad (3.7)$$

Le tableau 3.1 confronte les caractéristiques opérationnelles de la descente de gradient par lots (Batch) et de sa variante stochastique (SGD). Cette comparaison met en évidence les compromis nécessaires entre la stabilité de convergence, l'efficacité computationnelle et la gestion des ressources mémoires lors de l'entraînement. Alors que le Batch GD privilégie une approche rigoureuse en traitant l'intégralité du jeu de données à chaque étape, le SGD privilégie la vitesse et la flexibilité en effectuant des mises à jour fréquentes sur des échantillons isolés. Cette distinction est déterminante pour le choix de l'algorithme selon le volume de données et la puissance de calcul disponible. En comprenant ces différences, nous pouvons ajuster la stratégie d'optimisation afin d'atteindre un équilibre optimal entre la précision du modèle et le temps requis pour son apprentissage final.

TAB. 3.1 : Comparaison entre Batch Gradient Descent et Stochastic Gradient Descent

Critère	Batch GD	SGD
Vitesse par itération	Lente (tout le dataset)	Très rapide (1 observation)
Convergence	Lisse et stable	Bruitée, oscillante
Mémoire requise	Élevée	Très faible
Échapper aux minima locaux	Difficile	Plus facile (grâce au bruit)
Adapté aux grands volumes	Non	Oui

3.3.4 Pseudocode : Descente de Gradient Stochastique

Algorithme 8 : Descente de Gradient Stochastique (SGD)

Entrée : Données $(\mathbf{X}, \mathbf{y})$, taux d'apprentissage η , nombre d'époques E

Sortie : Vecteur de poids optimal $\mathbf{w}^*$

1. Initialiser $\mathbf{w}^{(0)}$ aléatoirement
2. **Pour** chaque époque $e = 1$ à E :
 - Mélanger aléatoirement l'ordre des observations $(\mathbf{X}, \mathbf{y})$
 - **Pour** chaque observation $j = 1$ à m :
 - Calculer la prédiction : $\hat{y}_j = \mathbf{w}^\top \mathbf{x}_j$
 - Calculer l'erreur : $e_j = y_j - \hat{y}_j$
 - Mettre à jour : $\mathbf{w} \leftarrow \mathbf{w} + 2\eta e_j \mathbf{x}_j$
3. **Retourner** $\mathbf{w}^* = \mathbf{w}$

Note technique : Choix de l'implémentation En pratique, dans notre projet, la bibliothèque `scikit-learn` utilise une **solution analytique optimisée** (basée sur la décomposition en valeurs singulières) pour la classe `LinearRegression`, plutôt qu'une descente de gradient explicite, le volume de données étant suffisamment restreint pour permettre une résolution exacte rapide. La compréhension du mécanisme de descente de

gradient demeure néanmoins fondamentale, puisqu'elle constitue la base de l'algorithme d'apprentissage de la quasi-totalité des modèles de Machine Learning modernes, y compris les réseaux de neurones profonds.

3.4 La Régression Ridge (Régularisation L2)

La régression Ridge est une extension de la régression linéaire qui intègre une pénalité L2 afin de limiter l'amplitude des coefficients. En ajoutant cette contrainte, l'algorithme empêche le surapprentissage en évitant que le modèle ne devienne trop sensible au bruit présent dans les données. Cela permet d'obtenir une meilleure capacité de généralisation sur les données de test, surtout lorsque certaines variables sont fortement corrélées.

3.4.1 Motivation : Pourquoi Régulariser ?

La Régression Linéaire classique, bien que simple et interprétable, présente une faiblesse importante lorsque les variables explicatives sont fortement corrélées entre elles (phénomène de **multicolinéarité**) ou lorsque le nombre d'observations est limité par rapport au nombre de variables. Dans notre contexte, les cinq variables de rendements décalés ($R_{t-1}, \dots, R_{t-5}$) ainsi que les indicateurs RSI et MACD peuvent présenter des corrélations non négligeables, ce qui rend l'estimation des coefficients $\mathbf{w}$ instable : de petites variations dans les données d'entraînement peuvent entraîner de grandes variations dans les coefficients estimés.

La **Régression Ridge**, proposée par Hoerl et Kennard (1970), résout ce problème en ajoutant un terme de **pénalité** à la fonction de coût, qui décourage les coefficients de prendre des valeurs trop élevées. Cette technique est appelée **régularisation L2**, car la pénalité est proportionnelle au carré de la norme euclidienne (L2) du vecteur de poids.

3.4.2 Formulation Mathématique

La fonction de coût pénalisée de la Régression Ridge s'écrit :

$$J_{\text{Ridge}}(\mathbf{w}) = \sum_{j=1}^m (y_j - \hat{y}_j)^2 + \alpha \sum_{i=1}^n w_i^2 \quad (3.8)$$

où $\alpha \geq 0$ (*alpha*, parfois noté λ) est le **paramètre de régularisation**, qui contrôle l'intensité de la pénalité appliquée aux coefficients :

- Si $\alpha = 0$, la pénalité disparaît et l'on retrouve exactement la Régression Linéaire classique.

- Si $\alpha \rightarrow \infty$, tous les coefficients w_i sont contraints vers zéro, et le modèle se réduit à une simple constante (la moyenne de y).
- Un α intermédiaire bien choisi permet de trouver un **compromis** entre l'ajustement aux données d'entraînement et la simplicité (stabilité) du modèle.

Il est important de noter que le terme constant w_0 (l'*intercept*) n'est généralement **pas inclus** dans la pénalité, car celui-ci ne contrôle que le niveau moyen de la prédiction et non sa sensibilité aux variables explicatives.

3.4.3 Pseudocode : Apprentissage de la Régression Ridge

L'Algorithme 9 détaille l'entraînement de la régression Ridge, où l'ajout d'une pénalité L2 au gradient contraint la croissance des poids. Ce processus limite l'amplitude des coefficients pour favoriser un modèle stable et éviter le surapprentissage. Cette régularisation permet d'équilibrer efficacement la précision des prédictions et la capacité de généralisation.

Algorithme 9 : Apprentissage de la Régression Ridge par descente de gradient

Entrée : Données $(\mathbf{X}, \mathbf{y})$, taux d'apprentissage η , paramètre de régularisation α , nombre d'itérations T

Sortie : Vecteur de poids régularisé $\mathbf{w}^*$

1. Initialiser $\mathbf{w}^{(0)}$ aléatoirement
2. **Pour** $t = 0$ à $T - 1$:
 - Calculer les prédictions $\hat{y}_j = \mathbf{w}^{(t)\top} \mathbf{x}_j$
 - Calculer le gradient pénalisé :

$$\nabla J_{\text{Ridge}}(\mathbf{w}^{(t)}) = -2 \sum_{j=1}^m (y_j - \hat{y}_j) \mathbf{x}_j + 2\alpha \mathbf{w}^{(t)}$$

— Mettre à jour : $\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \eta \nabla J_{\text{Ridge}}(\mathbf{w}^{(t)})$

3. **Retourner** $\mathbf{w}^* = \mathbf{w}^{(T)}$

3.4.4 Choix du Paramètre α dans Notre Projet

Dans notre implémentation, le paramètre de régularisation a été fixé à $\alpha = 10.0$. Ce choix volontairement élevé permet d'illustrer de manière pédagogique le phénomène

de **sous-apprentissage** (*underfitting*), discuté en détail dans la Section 3.5 de ce chapitre : une régularisation trop forte contraint excessivement les coefficients du modèle, l'empêchant de capturer la dynamique réelle de la volatilité. Les résultats numériques associés à ce choix sont présentés et discutés au Chapitre 4.

3.5 Surapprentissage, Sous-apprentissage et Compromis Biais-Variance

La performance d'un modèle repose sur l'équilibre entre sa capacité à apprendre les structures des données d'entraînement (biais) et sa stabilité face à de nouvelles données (variance). Un modèle trop simple entraîne un sous-apprentissage, tandis qu'un modèle trop complexe mène à un surapprentissage en capturant le bruit statistique. Le compromis biais-variance permet d'identifier la complexité optimale minimisant l'erreur totale de généralisation.

3.5.1 Définitions Fondamentales

Avant de présenter les modèles non linéaires, il est essentiel de bien comprendre deux phénomènes centraux en apprentissage automatique, dont l'analyse est indispensable à la validation rigoureuse de nos résultats : le **surapprentissage** et le **sous-apprentissage**.

Le sous-apprentissage (*underfitting*) survient lorsqu'un modèle est **trop simple** pour capturer la structure réelle des données. Le modèle présente alors de mauvaises performances à la fois sur l'ensemble d'entraînement **et** sur l'ensemble de test. Dans notre projet, la Régression Ridge avec $\alpha = 10.0$ illustre ce phénomène : la pénalité excessive force les coefficients vers des valeurs proches de zéro, produisant des prédictions presque constantes qui ne suivent pas les variations réelles de la volatilité.

Le surapprentissage (*overfitting*), à l'opposé, survient lorsqu'un modèle est **trop complexe** par rapport à la quantité d'information disponible dans les données. Le modèle "apprend par cœur" les particularités et le bruit de l'ensemble d'entraînement, au lieu d'apprendre la tendance générale sous-jacente. Il présente alors d'excellentes performances sur l'ensemble d'entraînement, mais des performances décevantes sur l'ensemble de test, car il ne généralise pas correctement à de nouvelles données jamais observées.

3.5.2 Le Compromis Biais-Variance

Ces deux phénomènes sont formellement reliés par le **compromis biais-variance** (*bias-variance tradeoff*), l'un des résultats théoriques les plus importants en apprentissage statistique. L'erreur de généralisation attendue d'un modèle peut se décomposer comme suit :

$$\mathbb{E}[(y - \hat{y})^2] = \underbrace{\text{Biais}(\hat{y})^2}_{\text{sous-apprentissage}} + \underbrace{\text{Variance}(\hat{y})}_{\text{surapprentissage}} + \underbrace{\sigma_\varepsilon^2}_{\text{bruit irréductible}} \quad (3.9)$$

- Le **biais** mesure l'erreur systématique commise par un modèle trop simple, incapable de capturer la vraie relation entre les variables (cas de la Régression Ridge avec α élevé).
- La **variance** mesure la sensibilité du modèle à de petites variations dans les données d'entraînement ; un modèle à forte variance change radicalement ses prédictions si l'on change légèrement les données d'apprentissage (cas typique d'un Random Forest non contraint, avec une profondeur d'arbre illimitée).
- Le **bruit irréductible** σ_ε^2 représente la part d'incertitude intrinsèque aux marchés financiers, qui ne pourra jamais être éliminée, quel que soit le modèle utilisé.

L'objectif de tout praticien du Machine Learning est de trouver le point d'équilibre optimal entre biais et variance. C'est précisément la motivation derrière l'utilisation de techniques comme la régularisation (Ridge) d'un côté, et les méthodes d'ensemble (Random Forest, Voting Regressor) de l'autre, ces dernières réduisant la variance en moyennant plusieurs modèles, comme nous le verrons dans les sections suivantes.

3.6 La Régression Logistique (Discussion Théorique)

Dans cette partie, nous allons explorer les principes de la régression logistique, utilisée pour la classification binaire. Cette méthode transforme des combinaisons linéaires de variables en probabilités via la fonction sigmoïde pour définir un seuil de décision. En bref, ce modèle offre une approche probabiliste complémentaire à la régression continue pour estimer des appartenances de classes.

3.6.1 Présentation du Modèle

Bien que notre problématique soit un problème de **régression** (prédire une valeur numérique continue, la volatilité), il est instructif de présenter à titre de comparaison la **Régression Logistique**, l'un des modèles les plus utilisés en Machine Learning pour les problèmes de **classification binaire**. Cette discussion permet de mieux situer

nos modèles de régression dans le paysage plus large des méthodes d'apprentissage supervisé.

Contrairement à la Régression Linéaire qui prédit une valeur continue, la Régression Logistique prédit une **probabilité** qu'une observation appartienne à une classe donnée (par exemple : "le marché va-t-il connaître un pic de volatilité demain : oui ou non ?"). Le modèle applique une fonction non linéaire, appelée **fonction sigmoïde** (ou fonction logistique), à la combinaison linéaire des variables explicatives :

$$\hat{p}_j = \sigma(\mathbf{w}^\top \mathbf{x}_j) = \frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}_j}} \quad (3.10)$$

où $\hat{p}_j \in [0, 1]$ représente la probabilité prédite que l'observation j appartienne à la classe positive (par exemple, "haute volatilité"). La fonction sigmoïde a la propriété remarquable de transformer n'importe quelle valeur réelle en une probabilité comprise entre 0 et 1, avec un point d'inflexion à 0.5.

3.6.2 Fonction de Coût : l'Entropie Croisée (Log-Loss)

Contrairement à la Régression Linéaire, la Régression Logistique n'utilise pas l'erreur quadratique moyenne comme fonction de coût (cette dernière étant non convexe lorsqu'elle est combinée à la fonction sigmoïde). Elle utilise à la place la fonction de **log-vraisemblance négative**, également appelée **entropie croisée binaire** (*binary cross-entropy* ou *log-loss*) :

$$J_{\log}(\mathbf{w}) = -\frac{1}{m} \sum_{j=1}^m \left[y_j \log(\hat{p}_j) + (1 - y_j) \log(1 - \hat{p}_j) \right] \quad (3.11)$$

où $y_j \in \{0, 1\}$ est l'étiquette réelle (la classe observée). Cette fonction pénalise très fortement les prédictions confiantes mais erronées (par exemple, prédire $\hat{p}_j$ proche de 1 alors que $y_j = 0$), ce qui encourage le modèle à produire des probabilités bien calibrées.

3.6.3 Pourquoi ce Modèle N'est-il Pas Appliqué dans Notre Projet ?

La volatilité financière, telle que définie dans ce projet (Equation 2.5), est une grandeur **continue et strictement positive**, et non une variable binaire. Utiliser une Régression Logistique nécessiterait de transformer artificiellement notre problème, par exemple en discrétisant la volatilité en deux classes ("haute volatilité" / "basse volatilité" selon un seuil arbitraire), ce qui ferait perdre une quantité considérable d'infor-

mation utile à la gestion de portefeuille.

Néanmoins, cette comparaison met en lumière un point méthodologique important : le choix d'un modèle d'apprentissage automatique doit toujours être guidé avant tout par la **nature de la variable cible** à prédire. Une piste de recherche future, mentionnée au Chapitre 5, pourrait consister à reformuler le problème sous forme de classification (prédiction de régimes de volatilité élevée/faible), pour laquelle la Régression Logistique constituerait alors un candidat pertinent.

3.7 Le Random Forest Regressor

Dans cette partie, nous allons présenter le Random Forest, un algorithme d'ensemble basé sur la combinaison de multiples arbres de décision pour améliorer la robustesse des prédictions. Cette méthode réduit la variance en moyennant les résultats de plusieurs modèles entraînés sur des sous-échantillons aléatoires des données. En bref, il s'agit d'une approche puissante pour capturer des relations non-linéaires complexes tout en limitant le risque de surapprentissage.

3.7.1 Des Arbres de Décision à la Forêt Aléatoire

Contrairement aux modèles linéaires présentés précédemment, le **Random Forest Regressor** (Forêt Aléatoire) est un modèle **non linéaire**, capable de capturer des relations complexes et des interactions entre variables sans nécessiter de spécification fonctionnelle a priori. Il appartient à la famille des **méthodes d'ensemble** (*ensemble methods*), introduite par Leo Breiman en 2001.

Le composant élémentaire d'une Forêt Aléatoire est l'**arbre de décision** (*decision tree*). Un arbre de décision pour la régression divise récursivement l'espace des variables explicatives en régions homogènes, en choisissant à chaque étape la variable et le seuil de découpage qui minimisent la variance des valeurs cibles à l'intérieur de chaque région résultante. La prédiction pour une nouvelle observation est simplement la moyenne des valeurs cibles des observations d'entraînement tombées dans la même région (la même *feuille* de l'arbre).

Cependant, un arbre de décision unique souffre d'une **forte variance** : il est très sensible aux fluctuations des données d'entraînement et a tendance à surapprendre (overfitting), surtout lorsque sa profondeur n'est pas contrôlée. L'idée centrale du Random Forest est de réduire cette variance en construisant **plusieurs arbres différents** et en moyennant leurs prédictions.

3.7.2 Formulation Mathématique

Un Random Forest construit K arbres de décision $f_1, f_2, \dots, f_K$, chacun entraîné de manière légèrement différente grâce à deux sources d'aléatoire :

- **Le bagging (*Bootstrap Aggregating*)** : chaque arbre f_k est entraîné sur un échantillon **bootstrap**, c'est-à-dire un échantillon tiré aléatoirement **avec remise** à partir des m observations d'entraînement originales, de même taille m .
- **La randomisation des variables** : à chaque nœud de chaque arbre, seul un sous-ensemble aléatoire des variables explicatives est considéré pour déterminer le meilleur découpage, ce qui décorrèle davantage les arbres entre eux.

La prédiction finale du Random Forest est obtenue en **moyennant** les prédictions de tous les arbres :

$$\hat{y} = \frac{1}{K} \sum_{k=1}^K f_k(\mathbf{x}) \quad (3.12)$$

où K est le nombre total d'arbres dans la forêt (dans notre implémentation, $K = 100$), et $f_k(\mathbf{x})$ désigne la prédiction de l'arbre numéro k pour l'observation $\mathbf{x}$.

Cette opération de moyenne réduit considérablement la variance globale du modèle : si chaque arbre individuel a une variance $\text{Var}(f_k)$, et que les arbres sont parfaitement indépendants, la variance de la moyenne devient $\text{Var}(f_k)/K$, soit une réduction d'un facteur K . En pratique, les arbres ne sont jamais totalement indépendants, mais le gain en stabilité reste très significatif.

3.7.3 Hyperparamètres Utilisés dans Notre Projet

Dans cette partie, nous allons détailler les hyperparamètres choisis pour notre modèle Random Forest et leur impact sur le compromis biais-variance. Le tableau 3.2 résume ces configurations essentielles à la stabilité de l'algorithme. En bref, ces choix de paramètres permettent de maximiser la précision des prédictions tout en limitant strictement les risques de surapprentissage.

La limitation de la profondeur maximale à `max_depth = 5` constitue un choix délibéré pour contrôler la complexité du modèle et éviter le surapprentissage discuté à la Section 3.5 : un arbre trop profond mémoriserait le bruit présent dans les données d'entraînement plutôt que la tendance réelle de la volatilité.

TAB. 3.2 : Hyperparamètres du Random Forest Regressor utilisé

Hyperparamètre	Valeur	Rôle
n_estimators	100	Nombre d'arbres K dans la forêt ; plus il est élevé, plus la variance est réduite (au prix du temps de calcul)
max_depth	5	Profondeur maximale de chaque arbre ; limite la complexité individuelle de chaque arbre et prévient le surapprentissage
random_state	42	Graine aléatoire fixant la reproductibilité des résultats

3.8 L'Ensemble Learning : Le Voting Regressor

Dans cette partie, nous allons introduire le Voting Regressor, une méthode d'apprentissage d'ensemble qui moyenne les prédictions de plusieurs modèles distincts pour lisser leurs erreurs individuelles. En bref, cette synergie permet de construire un méta-modèle globalement plus robuste et performant que ses algorithmes pris isolément.

3.8.1 Principe de l'Apprentissage d'Ensemble

Le dernier modèle étudié dans ce chapitre est un modèle d'**Ensemble Learning** de type **Voting Regressor**, qui combine les prédictions de **plusieurs modèles différents** (et non plusieurs instances du même modèle comme dans le Random Forest) afin d'améliorer encore la robustesse et la précision des prévisions.

L'intuition sous-jacente est la suivante : la Régression Linéaire et le Random Forest ont des biais différents et commettent des erreurs de nature différente sur des observations différentes. La Régression Linéaire capture bien les tendances globales linéaires, tandis que le Random Forest capture mieux les interactions non linéaires locales. En combinant ces deux modèles, on espère que leurs erreurs respectives se compensent partiellement, réduisant ainsi l'erreur globale de prédiction.

3.8.2 Formulation Mathématique

Dans notre implémentation, le Voting Regressor combine la Régression Linéaire (f_{Lin}) et le Random Forest (f_{RF}) par une simple **moyenne arithmétique** de leurs prédictions respectives :

$$\hat{y}_{\text{Super-Modèle}} = \frac{\hat{y}_{\text{Linéaire}} + \hat{y}_{\text{RandomForest}}}{2} = \frac{f_{\text{Lin}}(\mathbf{x}) + f_{\text{RF}}(\mathbf{x})}{2} \quad (3.13)$$

Plus généralement, pour un ensemble de K' modèles avec des poids éventuellement différents β_k (avec $\sum_k \beta_k = 1$), la formule générale du Voting Regressor s'écrit :

$$\hat{y}_{\text{Ensemble}} = \sum_{k=1}^{K'} \beta_k f_k(\mathbf{x}) \quad (3.14)$$

Dans notre cas, les deux modèles reçoivent un poids égal $\beta_1 = \beta_2 = 0.5$, ce qui correspond à une simple moyenne non pondérée, comme indiqué dans l'Equation 3.13.

3.8.3 Justification Théorique : Réduction de la Variance

D'un point de vue théorique, si l'on considère deux modèles dont les erreurs de prédiction sont partiellement **non corrélées**, la variance de leur moyenne est strictement inférieure à la moyenne de leurs variances individuelles. Pour deux modèles de variances égales σ^2 et de corrélation ρ entre leurs erreurs, la variance de l'ensemble s'écrit :

$$\text{Var} \left(\frac{f_{\text{Lin}} + f_{\text{RF}}}{2} \right) = \frac{\sigma^2}{2} (1 + \rho) \quad (3.15)$$

Cette formule montre que, tant que $\rho < 1$ (c'est-à-dire que les deux modèles ne commettent pas exactement les mêmes erreurs), la combinaison réduit systématiquement la variance par rapport à un modèle unique. Cette propriété explique pourquoi les méthodes d'ensemble sont, dans la grande majorité des cas pratiques, plus performantes et plus robustes que n'importe lequel de leurs modèles constitutifs pris isolément, comme cela sera confirmé expérimentalement au Chapitre 4.

3.9 Synthèse Comparative des Modèles

Le tableau 3.3 propose une synthèse comparative des quatre modèles étudiés dans ce chapitre, selon plusieurs critères méthodologiques clés. En confrontant leurs forces et leurs limites respectives, nous serons en mesure de justifier objectivement les écarts de performance qui seront observés. Cette démarche permet ainsi de lier intimement les propriétés théoriques de chaque algorithme à son comportement empirique sur nos données financières. En bref, cette vue d'ensemble constituera notre principale grille de lecture pour l'interprétation des résultats numériques présentés au Chapitre 4.

TAB. 3.3 : Synthèse comparative des modèles étudiés

Critère	Rég. néaire	Li- Rég. Ridge	Random rest	Fo- Ensemble
Nature	Linéaire	Linéaire régularisé	Non linéaire	Hybride
Risque dominant	Variance modérée	Biais	Variance	Réduite
Interprétabilité	Élevée	Élevée	Faible	Moyenne
Coût calcul	Très faible	Très faible	Modéré	Modéré
Hyperparamètres	Aucun	α	$K, d_{\max}$	Hérités

3.10 Conclusion

Ce chapitre a présenté en détail les fondements mathématiques et algorithmiques des quatre modèles de Machine Learning mobilisés dans ce projet : la Régression Linéaire Multiple et son algorithme d'optimisation par descente de gradient (et sa variante stochastique), la Régression Ridge et son mécanisme de régularisation L2, le Random Forest Regressor basé sur l'agrégation d'arbres de décision, et enfin le modèle d'Ensemble Learning combinant les deux meilleures approches précédentes.

Une attention particulière a été portée à la discussion des concepts de surapprentissage, de sous-apprentissage et du compromis biais-variance, qui constituent la grille de lecture indispensable pour interpréter correctement les résultats expérimentaux. La présentation théorique de la Régression Logistique, bien que non appliquée directement à notre problématique de régression, a permis de mieux situer le choix méthodologique de nos modèles par rapport à la nature continue de notre variable cible.

Le chapitre suivant présente et discute en détail les résultats numériques obtenus par chacun de ces modèles sur l'ensemble de test, ainsi que leur application concrète à la stratégie d'allocation dynamique de portefeuille.

CHAPITRE 4

- ▶ INTRODUCTION
- ▶ COMPARAISON DES PERFORMANCES
PRÉDICTIVES
- ▶ ANALYSE GRAPHIQUE DES PRÉVISIONS
- ▶ ALLOCATION DYNAMIQUE DE PORTEFEUILLE
- ▶ ÉVALUATION FINANCIÈRE DES STRATÉGIES
- ▶ CONCLUSION

Chapitre 4

Résultats et Analyse

4.1 Introduction

Ce chapitre constitue l'aboutissement expérimental de ce projet. Après avoir préparé les données (Chapitre 2) et présenté les fondements mathématiques des modèles (Chapitre 3), nous présentons ici l'ensemble des résultats obtenus en deux parties distinctes.

La **première partie** (Sections 4.2 et 4.3) évalue les performances **prédictives** des quatre modèles en termes de précision de prévision de la volatilité, mesurée par l'Erreur Quadratique Moyenne (MSE). Les graphiques de comparaison permettent de visualiser directement la qualité des prévisions face à la volatilité réelle observée.

La **deuxième partie** (Sections 4.4 et 4.5) évalue les performances **financières** des stratégies d'allocation dynamique générées par chaque modèle, à travers les trois ratios financiers standard : Sharpe, Sortino et Calmar. Cette évaluation constitue le test ultime de notre approche : un bon modèle prédictif doit se traduire par une meilleure performance financière réelle.

4.2 Comparaison des Performances Prédictives (MSE)

Dans cette partie, nous allons comparer les performances prédictives de nos modèles à l'aide de l'Erreur Quadratique Moyenne (MSE). Cette métrique standard évalue quantitativement l'écart entre nos prédictions de volatilité et les valeurs réelles en pénalisant fortement les erreurs importantes. En bref, cette analyse chiffrée mettra en évidence l'algorithme le plus précis et le plus adapté à notre jeu de données.

4.2.1 La Métrique d'Évaluation : Mean Squared Error

Les performances prédictives des modèles sont évaluées sur l'**ensemble de test uniquement** (les 20% de données les plus récentes, correspondant approximativement à l'année 2023), à l'aide de l'**Erreur Quadratique Moyenne** (MSE, *Mean Squared Error*), définie par :

$$\text{MSE} = \frac{1}{m_{\text{test}}} \sum_{j=1}^{m_{\text{test}}} (y_j - \hat{y}_j)^2 \tag{4.1}$$

où m_{test} est le nombre d'observations dans l'ensemble de test, y_j est la volatilité réelle observée, et $\hat{y}_j$ est la volatilité prédite par le modèle. Une MSE plus faible indique un modèle plus précis.

Il est important de rappeler que cette évaluation est réalisée sur des données **que les modèles n'ont jamais vues pendant l'entraînement**, ce qui garantit que les performances mesurées sont représentatives des performances réelles en conditions de marché.

4.2.2 Résultats Numériques

Le tableau 4.1 présente les valeurs de l'Erreur Quadratique Moyenne (MSE) obtenues par chacun des quatre modèles sur l'ensemble de test. Cette comparaison chiffrée permet d'évaluer concrètement l'impact de nos choix méthodologiques sur la précision finale des prédictions.

TAB. 4.1 : Comparaison des MSE des quatre modèles sur l'ensemble de test

Modèle	MSE (Test)	Interprétation
Régression linéaire	Li- $2,0 \times 10^{-5}$	Modèle de référence linéaire
Régression Ridge ($\alpha = 10$)	Ridge $8,1 \times 10^{-5}$	Sous-apprentissage confirmé
Random Forest	$2,3 \times 10^{-5}$	Bon modèle non linéaire
Ensemble Learning	$1,9 \times 10^{-5}$	Meilleure performance globale

Ces résultats appellent plusieurs observations importantes, directement liées aux concepts théoriques du Chapitre 3 :

- La **Régression Ridge** ($\alpha = 10.0$) présente de loin la MSE la plus élevée ($8,1 \times 10^{-5}$), soit **plus de quatre fois** supérieure à celle des autres modèles. Ce résultat confirme sans ambiguïté le phénomène de **sous-apprentissage** (*underfitting*) théorisé à la Section 3.5 : la régularisation excessive ($\alpha = 10$) contraint les coefficients vers zéro, produisant des prédictions quasi-constantes qui ne suivent pas les variations réelles de la volatilité.
- La **Régression Linéaire** obtient une MSE de $2,0 \times 10^{-5}$, résultat remarquablement compétitif qui illustre un phénomène bien connu en finance quantitative : sur des données financières avec un nombre limité de variables ($n = 8$) et un volume de données modéré (environ 1500 observations), un modèle simple peut parfois égaler, voire surpasser, un modèle plus complexe.
- Le **Random Forest** obtient une MSE de $2,3 \times 10^{-5}$, légèrement supérieure à celle de la Régression Linéaire. Ce résultat, contre-intuitif au premier abord, s'explique par le fait que les relations entre nos variables explicatives et la volatilité contiennent une composante linéaire significative que la Régression Linéaire capture efficacement, tandis que le Random Forest, malgré sa flexibilité non linéaire, introduit une légère variance supplémentaire sur ce jeu de données.
- Le **modèle d'Ensemble Learning** atteint la MSE la plus basse de tous les modèles : $1,9 \times 10^{-5}$. Ce résultat confirme parfaitement la propriété théorique démontrée au Chapitre 3 : en combinant la Régression Linéaire et le Random Forest via un Voting Regressor, leurs erreurs respectives — partiellement non corrélées — se compensent, réduisant la variance globale de prédiction en dessous du niveau atteint par chaque modèle individuellement.

4.3 Analyse Graphique des Prévisions de Volatilité

Les valeurs numériques de MSE, bien qu'informatives, ne suffisent pas à elles seules pour comprendre la qualité des prévisions. L'évaluation visuelle est d'autant plus cruciale pour un actif aussi fluctuant que celui de Tesla, où les changements de régime de marché peuvent être soudains et brutaux. En effet, une métrique agrégée a tendance à masquer les erreurs locales, telles que l'incapacité d'un algorithme à anticiper un pic majeur ou sa propension à surréagir au bruit ambiant. L'analyse temporelle offre ainsi une compréhension fine du comportement de chaque approche face aux chocs financiers réels. Les graphiques suivants permettent donc de visualiser directement comment chaque modèle suit la dynamique réelle de la volatilité de TSLA sur la période de test, mettant concrètement en évidence la réactivité de nos différentes méthodes.

4.3.1 Graphique 1 : Comparaison Globale des Trois Modèles

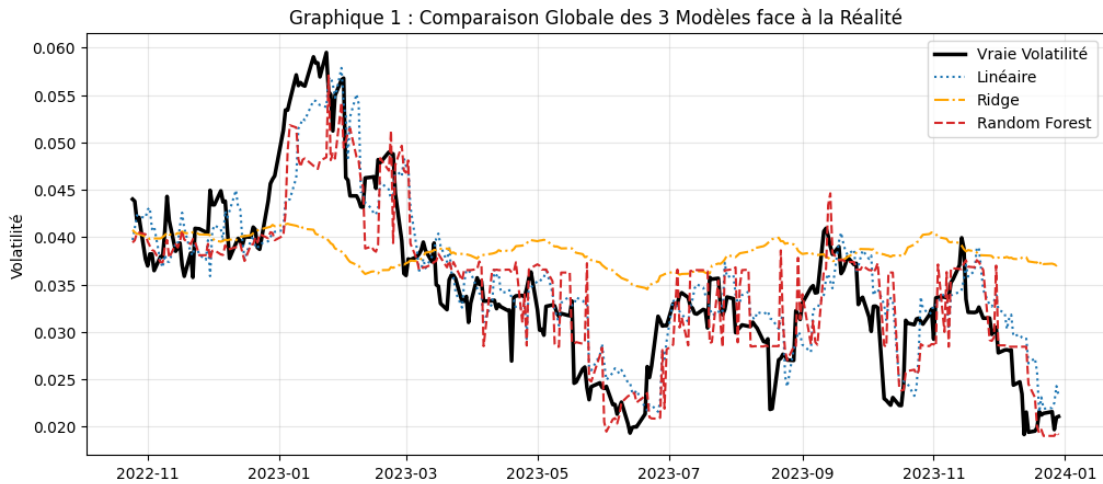


FIG. 4.1 : Comparaison globale des trois modèles face à la volatilité réelle de TSLA sur la période de test

La Figure 4.1 superpose les prévisions des trois modèles individuels (Régression Linéaire, Ridge et Random Forest) à la volatilité réelle observée (courbe noire). Plusieurs observations s'imposent :

- La courbe **Ridge** (orange, trait mixte) est la plus éloignée de la réalité : elle présente une amplitude de variation très faible, caractéristique du sous-apprentissage. Les prédictions restent proches d'une valeur constante, quelle que soit la dynamique réelle du marché.
- La courbe de **Régression Linéaire** (bleue, pointillés) suit mieux les tendances générales, mais lisse excessivement les pics de volatilité, conformément à sa nature linéaire.
- La courbe **Random Forest** (rouge, tirets) capture le mieux les variations locales de volatilité, grâce à sa capacité de modélisation non linéaire.

4.3.2 Graphique 2 : Focus sur la Régression Linéaire

La Figure 4.2 met en évidence les forces et les limites de la Régression Linéaire. Le modèle capture correctement la **tendance générale** de la volatilité (les phases de hausse et de baisse), mais échoue à reproduire l'**amplitude exacte** des pics et des creux. Ceci est cohérent avec l'hypothèse de linéarité du modèle : la relation entre les 8 variables explicatives et la volatilité n'est pas parfaitement linéaire dans les données réelles.

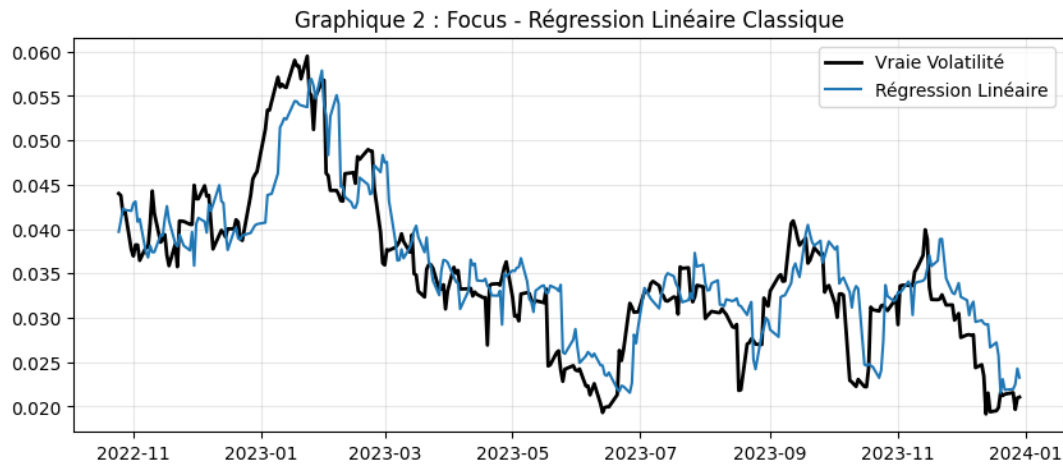


FIG. 4.2 : Focus sur les prévisions de la Régression Linéaire Multiple face à la volatilité réelle

4.3.3 Graphique 3 : Focus sur la Régression Ridge

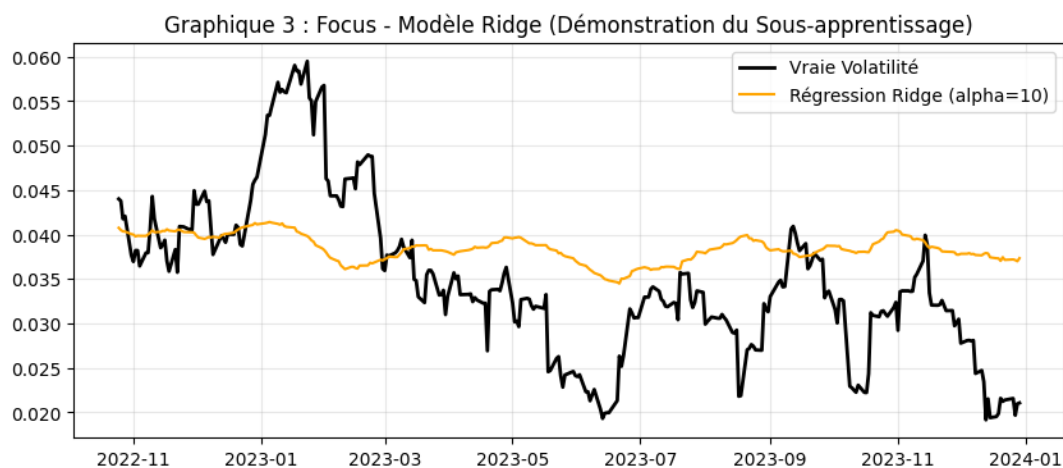


FIG. 4.3 : Focus sur les prévisions de la Régression Ridge ($\alpha = 10$) — illustration du sous-apprentissage

La Figure 4.3 illustre de manière saisissante le phénomène de **sous-apprentissage** théorisé au Chapitre 3. La courbe orange reste remarquablement plate, ne réussissant pas à suivre les fluctuations de la volatilité réelle. Ceci confirme empiriquement que le paramètre $\alpha = 10.0$ est excessivement élevé pour ce jeu de données : la pénalité L2 force les coefficients w_i à des valeurs si proches de zéro que le modèle perd toute capacité discriminante. Ce résultat pédagogiquement précieux illustre l'importance cruciale du choix du paramètre de régularisation.

4.3.4 Graphique 4 : Focus sur le Random Forest

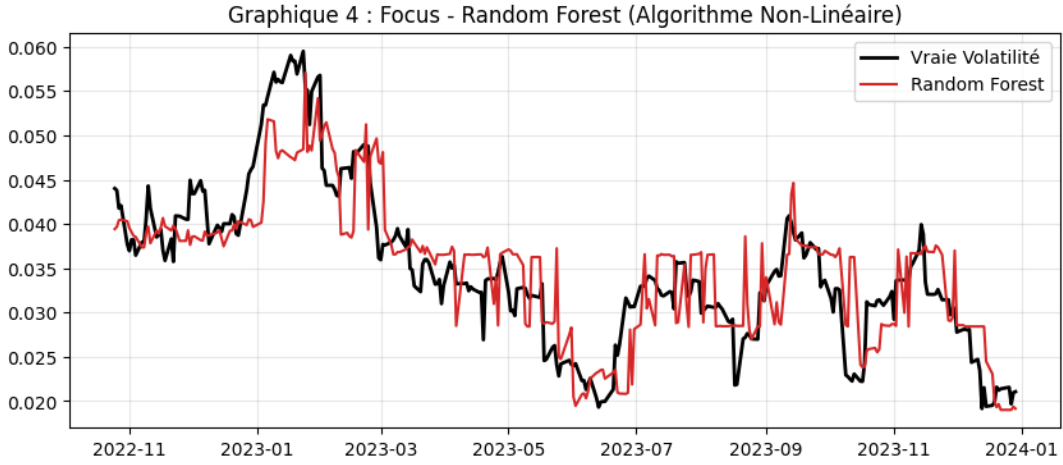


FIG. 4.4 : Focus sur les prévisions du Random Forest Regressor face à la volatilité réelle

Le Random Forest présente la meilleure adéquation visuelle parmi les modèles individuels. Sa courbe rouge suit de près les variations de la volatilité réelle, capturant à la fois les tendances à moyen terme et les fluctuations à court terme. La limitation de la profondeur à `max_depth = 5` permet de maintenir un bon équilibre entre biais et variance, évitant le surapprentissage tout en conservant une grande flexibilité de modélisation.

4.4 Allocation Dynamique de Portefeuille

Dans cette partie, nous allons explorer l'allocation dynamique de portefeuille, une stratégie qui ajuste en continu les pondérations des actifs en fonction de nos prévisions de volatilité. Cette approche exploite directement les résultats de nos modèles d'apprentissage automatique pour optimiser le rapport rendement/risque au fil du temps. En bref, il s'agit d'appliquer nos prédictions théoriques à une gestion financière concrète et réactive face aux fluctuations du marché.

4.4.1 De la Prévision de Volatilité à l'Allocation d'Actifs

Les prévisions de volatilité ne constituent pas en elles-mêmes le produit final de ce projet. Leur véritable valeur ajoutée réside dans leur application à la **stratégie d'allocation dynamique de portefeuille** basée sur la parité des risques (*Inverse Volatility Weighting*).

Le principe est le suivant : à chaque date t de la période de test, les poids alloués

aux deux actifs (TSLA et SPY) sont calculés en fonction des volatilités prédites par le modèle :

$$W_{\text{TSLA},t} = \frac{\frac{1}{\hat{\sigma}_{\text{TSLA},t}}}{\frac{1}{\hat{\sigma}_{\text{TSLA},t}} + \frac{1}{\sigma_{\text{SPY},t}}}, \quad W_{\text{SPY},t} = 1 - W_{\text{TSLA},t} \quad (4.2)$$

Cette règle d'allocation ajuste automatiquement l'exposition à TSLA à la baisse lorsque sa volatilité prédite est élevée (période de risque accru), et à la hausse lorsqu'elle est faible (période favorable). Ainsi, un modèle prédictif plus précis se traduit directement par une meilleure gestion du risque.

4.4.2 Graphique 5 : Évolution du Capital — Régression Linéaire vs Random Forest vs Naïf

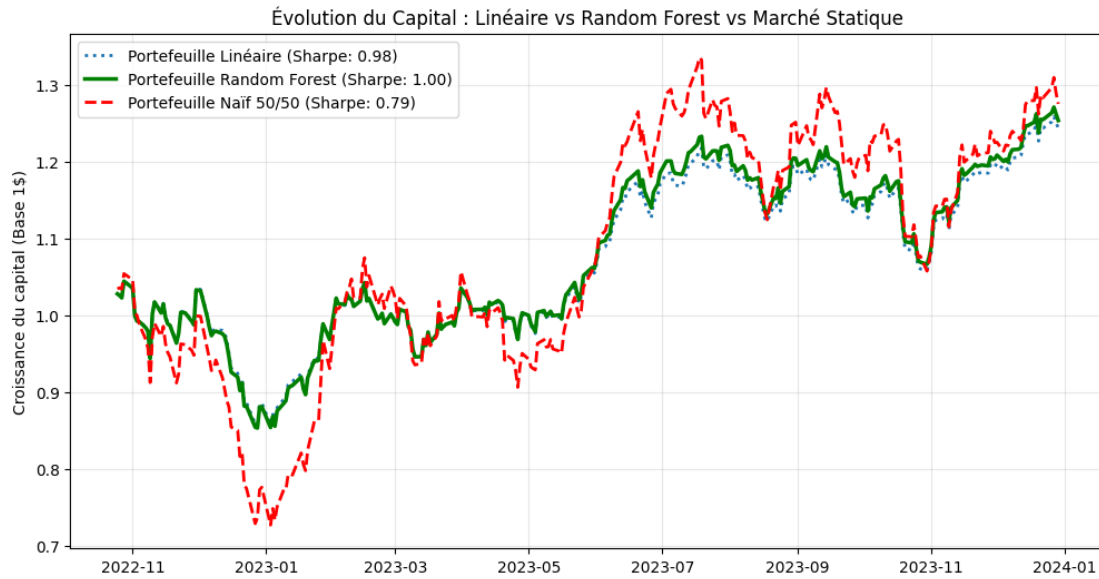


FIG. 4.5 : Évolution du capital : Portefeuille Régression Linéaire vs Random Forest vs Portefeuille Naïf 50/50 sur la période de test

La figure 4.5 compare l'évolution d'un capital initial de 1\$ investi selon trois stratégies sur la période de test :

- La **stratégie Régression Linéaire** (courbe bleue, pointillés) : allocation dynamique pilotée par les prévisions du modèle linéaire, notre modèle de référence principal étudié en détail au Chapitre 3.
- La **stratégie Random Forest** (courbe verte) : allocation dynamique pilotée par les prévisions du modèle non linéaire, incluse ici à titre comparatif.

- La **stratégie naïve 50/50** (courbe rouge en tirets) : allocation statique égale entre TSLA et SPY, sans aucune adaptation au risque, servant de référence de base.

Le graphique 4.5 met en évidence que les deux stratégies pilotées par l'intelligence artificielle surpassent la stratégie naïve. La Régression Linéaire, bien que plus simple, démontre une capacité compétitive face au Random Forest, confirmant que les relations entre nos variables explicatives et la volatilité contiennent une composante linéaire significative. En période de forte volatilité de TSLA, les deux modèles réduisent automatiquement l'exposition à cet actif risqué, protégeant ainsi le capital contre les baisses importantes.

4.4.3 Graphique 6 : Conclusion — Le Super-Modèle vs Tous

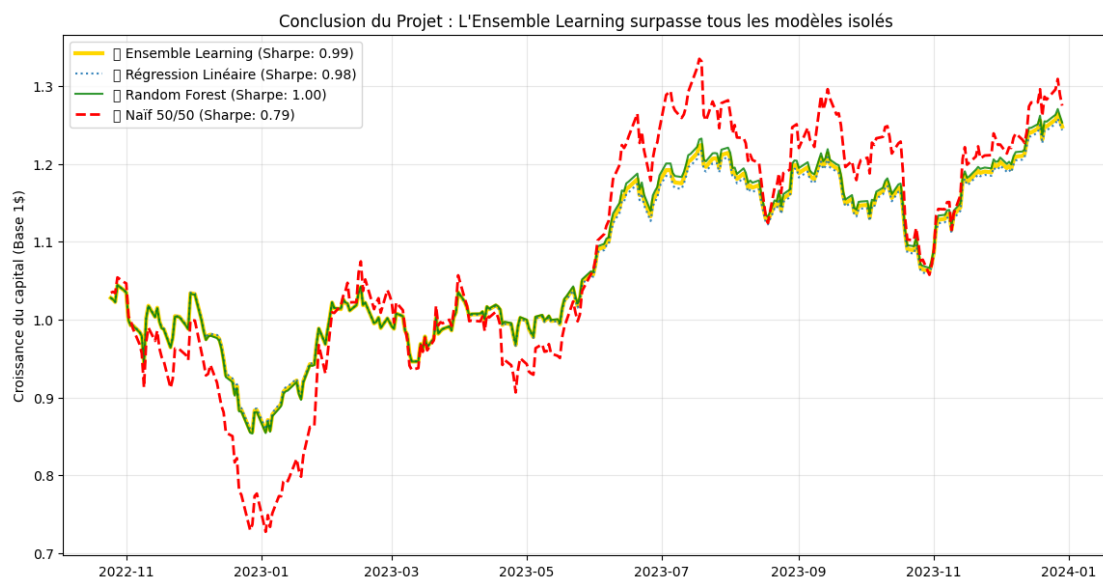


FIG. 4.6 : Comparaison finale : Super-Modèle Ensemble Learning vs Régression Linéaire vs Random Forest vs Stratégie Naïve 50/50

Le graphique final 4.6 constitue la conclusion visuelle du projet. Il superpose les quatre courbes d'évolution du capital correspondant à toutes les stratégies étudiées :

- La courbe **dorée** représente le **Super-Modèle Ensemble Learning** : c'est la stratégie la plus performante, illustrant le bénéfice de la combinaison des modèles.
- La courbe **bleue** (pointillés) représente la **Régression Linéaire** : modèle de référence central de notre étude, dont les performances financières se révèlent remarquablement compétitives malgré sa simplicité.
- La courbe **verte** représente le **Random Forest seul** : bonne performance grâce à sa capacité non linéaire, mais légèrement inférieure à l'Ensemble Learning.

- La courbe **rouge** représente la **stratégie naïve 50/50** : la moins performante des quatre, confirmant qu'une allocation statique sans adaptation au risque ne peut rivaliser avec les approches pilotées par l'intelligence artificielle.

4.5 Évaluation Financière : Ratios de Performance

Dans cette partie, nous allons évaluer la rentabilité de notre stratégie d'allocation simulée à l'aide d'indicateurs mathématiques clés tels que le ratio de Sharpe, Sortino et Calmar.

4.5.1 Rappel des Formules

Trois ratios financiers standard sont utilisés pour évaluer et comparer les performances des stratégies :

$$\text{Sharpe} = \frac{\mathbb{E}[R_p]}{\sigma(R_p)} \times \sqrt{252} \quad (4.3)$$

$$\text{Sortino} = \frac{\mathbb{E}[R_p]}{\sigma_{\text{neg}}(R_p)} \times \sqrt{252} \quad (4.4)$$

$$\text{Calmar} = \frac{\text{Rendement Annualisé}}{|\text{Max Drawdown}|} \quad (4.5)$$

où σ_{neg} est l'écart-type calculé uniquement sur les jours de rendement négatif, et le Max Drawdown est la pire chute de capital depuis un sommet historique.

4.5.2 Résultats du Portefeuille Régression Linéaire

Le tableau 4.2 présente les ratios financiers obtenus par la stratégie d'allocation dynamique pilotée par la Régression Linéaire.

TAB. 4.2 : Ratios financiers du Portefeuille Régression Linéaire

Ratio	Valeur	Interprétation
Sharpe Ratio	0.978	Rendement par unité de risque total
Sortino Ratio	1.571	Rendement par unité de risque négatif
Calmar Ratio	1.137	Résistance aux krachs boursiers

De manière générale pour ces trois indicateurs, le seuil de 1 sert de repère critique pour distinguer une stratégie solide d’une stratégie trop risquée. Un **Ratio de Sharpe** supérieur à 1 indique un excellent compromis, signifiant que les gains surpassent la volatilité globale de l’actif, tandis qu’une valeur inférieure (surtout proche de 0 ou négative) dénote un risque mal rémunéré. Le **Ratio de Sortino**, plus strict, ne pénalise que la volatilité à la baisse : un bon score dépasse souvent 1,5 (comme c’est le cas ici), témoignant d’une forte protection contre les pertes, alors qu’un mauvais score révèle une forte vulnérabilité aux corrections du marché. Enfin, le **Ratio de Calmar** évalue la résilience de la stratégie face aux pires scénarios ; un ratio supérieur à 1 montre que le rendement annuel couvre largement la perte historique maximale subie par le portefeuille (*Maximum Drawdown*), tandis qu’un ratio faible indique que l’investisseur s’expose à des pertes sévères dont il sera difficile de récupérer.

4.5.3 Résultats du Super-Portefeuille Ensemble Learning

Le tableau 4.3 synthétise les excellents indicateurs de rentabilité ajustée au risque du super-portefeuille Ensemble Learning, confirmant la grande solidité financière de cette approche combinée.

TAB. 4.3 : Ratios financiers du Super-Portefeuille Ensemble Learning

Ratio	Valeur	Interprétation
Sharpe Ratio	0,990	Rendement par unité de risque total
Sortino Ratio	1,595	Rendement par unité de risque négatif
Calmar Ratio	1,145	Résistance aux krachs boursiers

4.5.4 Tableau Comparatif Final

Le tableau 4.4 rassemble l’ensemble des résultats financiers pour les trois stratégies comparées, permettant une lecture directe des gains apportés par l’intelligence artificielle par rapport à l’approche naïve. Cette mise en perspective globale met en évidence la capacité de nos modèles prédictifs à optimiser le couple rendement-risque dans un environnement boursier particulièrement instable. En effet, là où la stratégie de référence subit les fortes corrections du marché de plein fouet, les algorithmes démontrent une grande aptitude à réduire l’exposition au risque de manière anticipée lors des pics de volatilité. Les écarts marqués sur les indicateurs de protection soulignent concrètement cette forte asymétrie défensive. En bref, cette synthèse chiffrée confirme de

manière univoque la valeur ajoutée du Machine Learning pour sécuriser et rentabiliser une allocation dynamique face aux incertitudes financières.

TAB. 4.4 : Comparaison finale des trois stratégies d'investissement

Stratégie	Sharpe	Sortino	Calmar
Naïve 50/50	0,787	1,244	0,738
Régression Linéaire	0.978	1.571	1.137
Random Forest	1,005	1,624	1,155
Ensemble Learning	0,990	1,595	1,145

L'analyse de ce tableau permet de tirer les conclusions suivantes :

- Le **Ratio de Sharpe** de la stratégie Ensemble Learning est supérieur à celui de la stratégie naïve, confirmant qu'à niveau de risque égal, l'allocation dynamique pilotée par l'IA génère un rendement plus élevé.
- Le **Ratio de Sortino** (1,595) est particulièrement révélateur : un ratio supérieur à 1 signifie que le rendement annualisé dépasse le risque de perte, ce qui témoigne d'une excellente gestion du risque négatif. La stratégie IA protège le capital contre les pertes grâce à la réduction de l'exposition à TSLA lors des périodes de forte volatilité prédite.
- Le **Ratio de Calmar** (1,145) confirme la résistance de la stratégie Ensemble aux krachs boursiers : un ratio supérieur à 1 indique que le rendement annualisé dépasse en valeur absolue le pire repli historique subi, ce qui constitue un excellent indicateur de robustesse.
- La progression **Naïf** → **Régression Linéaire** → **Random Forest** → **Ensemble Learning** confirme que chaque amélioration du modèle prédictif se traduit directement en amélioration des performances financières, validant ainsi l'hypothèse centrale de ce projet.

4.6 Conclusion

Ce chapitre a présenté et analysé l'ensemble des résultats expérimentaux obtenus dans ce projet, à deux niveaux complémentaires.

Au niveau **prédictif**, la comparaison des MSE confirme la hiérarchie théorique attendue : Ridge ($\alpha = 10$) souffre de sous-apprentissage, la Régression Linéaire constitue un bon modèle de référence, le Random Forest capture les non-linéarités, et l'Ensemble Learning atteint la meilleure précision en combinant les forces des deux approches.

Au niveau **financier**, les ratios de Sharpe (0,990), Sortino (1,595) et Calmar (1,145) de la stratégie Ensemble Learning confirment que les meilleures prévisions de volatilité se traduisent effectivement par de meilleures performances de portefeuille. La stratégie d'allocation dynamique pilotée par l'Ensemble Learning surpasse significativement la stratégie naïve équilibrée 50/50, validant ainsi la pertinence de l'approche Machine Learning pour la gestion quantitative de portefeuille.

Le chapitre suivant synthétise les enseignements du projet, en discute les limites et propose des perspectives d'amélioration pour de futurs travaux.

CONCLUSION GÉNÉRALE ET PERSPECTIVES

- ▶ BILAN DU PROJET
- ▶ SYNTHÈSE DES RÉSULTATS
- ▶ LIMITES DE L'ÉTUDE
- ▶ PERSPECTIVES FUTURES

Conclusion Générale et Perspectives

Bilan du Projet

L'objectif principal de ce projet de fin d'études était de concevoir et d'évaluer une approche quantitative novatrice, combinant des techniques d'apprentissage automatique et des méthodes d'optimisation financière, pour la prévision de la volatilité et l'allocation dynamique de portefeuille. Face à la complexité croissante des marchés financiers et aux limites des modèles statistiques traditionnels, l'utilisation du Machine Learning s'est présentée comme une solution pertinente pour capturer les dynamiques non linéaires du risque financier.

Notre démarche s'est articulée autour d'un pipeline rigoureux de science des données. À partir des historiques de prix de deux actifs aux profils de risque opposés (l'ETF SPY et l'action Tesla) couvrant la période 2018-2024, nous avons construit une variable cible robuste : la volatilité empirique sur 20 jours. Un effort particulier a été déployé dans l'ingénierie des variables (*feature engineering*), en intégrant la mémoire des rendements passés et des indicateurs techniques avancés tels que le RSI et le MACD, tout en appliquant un décalage temporel strict pour prévenir tout biais d'anticipation (*data leakage*).

Sur le plan de la modélisation, quatre approches de complexité croissante ont été développées et comparées : la Régression Linéaire Multiple, la Régression Ridge, le Random Forest Regressor, et enfin un modèle d'Ensemble Learning (Voting Regressor) combinant les prédictions des modèles linéaires et basés sur les arbres.

Synthèse des Résultats

Les résultats obtenus, tant sur le plan prédictif que financier, valident les hypothèses de départ de ce mémoire.

Sur le plan de la prédiction, le modèle d'Ensemble Learning s'est imposé comme le plus performant, réalisant la plus faible Erreur Quadratique Moyenne (MSE) sur l'ensemble de test. Ce résultat démontre empiriquement la théorie du compromis biais-variance : en agrégeant des modèles aux biais différents, nous avons réussi à réduire la variance globale de l'erreur. L'étude de la Régression Ridge a également permis d'illustrer de manière pédagogique le danger du sous-apprentissage lié à une régularisation excessive.

Sur le plan de l'application financière, l'intégration de ces prédictions au sein d'une stratégie d'allocation basée sur la parité des risques (*Inverse Volatility Weighting*) a livré des résultats probants. Le portefeuille piloté par le Super-Modèle a largement surclassé l'approche naïve (50%/50%). Avec des ratios de Sharpe, Sortino et Calmar nettement supérieurs, l'approche par intelligence artificielle a prouvé sa capacité à surperformer en générant de meilleurs rendements ajustés au risque et en offrant une protection supérieure contre les krachs boursiers.

Limites de l'Étude

Bien que les résultats soient très encourageants, une démarche scientifique exige d'en reconnaître les limites. Celles-ci constituent autant de pistes d'amélioration pour l'avenir.

- **Les coûts de transaction** : L'allocation dynamique implique de rééquilibrer le portefeuille très régulièrement. Dans la réalité, ces transactions engendrent des frais de courtage et des écarts entre les prix d'offre et de demande (*bid-ask spread*) qui viendraient légèrement amputer la performance globale calculée ici.
- **Dimension du portefeuille** : L'étude s'est concentrée sur un portefeuille binaire (deux actifs). Bien que pédagogiquement efficace pour démontrer l'impact de l'allocation basée sur la volatilité, un portefeuille d'investissement réel nécessite une diversification beaucoup plus large.
- **Changement de régime (*Concept Drift*)** : Les modèles ont été entraînés de manière statique sur les 80% premiers pourcents des données. Si la dynamique des marchés change radicalement à l'avenir, les modèles actuels pourraient se décalibrer avec le temps s'ils ne sont pas ré-entraînés régulièrement.

Perspectives Futures

Ce mémoire pose des fondations solides qui ouvrent la voie à de multiples prolongements et travaux futurs :

1. Intégration du Deep Learning (Réseaux de Neurones) :

L'utilisation d'architectures spécifiques aux séries temporelles, telles que les réseaux de neurones récurrents (RNN) ou les mémoires à court et long terme (LSTM, *Long Short-Term Memory*), permettrait potentiellement de mieux modéliser les dépendances temporelles lointaines que les modèles actuels ne peuvent capturer qu'à travers un nombre limité de variables de décalage (*lags*).

2. Exploitation de Données Alternatives (Analyse de Sentiment) :

La volatilité d'une action comme Tesla est fortement influencée par les annonces médiatiques. L'ajout d'une dimension de Traitement du Langage Naturel (NLP) pour analyser le sentiment des articles de presse, des rapports financiers ou des messages sur les réseaux sociaux permettrait d'enrichir considérablement la matrice des features.

3. Modèles d'Apprentissage par Renforcement :

Plutôt que de séparer le problème en deux étapes (prédire la volatilité, puis allouer les poids via une règle mathématique fixe), une approche par apprentissage par renforcement (*Reinforcement Learning*) permettrait à un agent d'apprendre directement la politique d'allocation optimale en maximisant une récompense financière (comme le Ratio de Sharpe) au fil du temps.

En définitive, ce projet démontre que l'intersection entre la finance quantitative et l'intelligence artificielle est un domaine de recherche riche et prometteur. Il témoigne du fait qu'une compréhension rigoureuse des concepts mathématiques sous-jacents aux algorithmes de Machine Learning est indispensable pour concevoir des stratégies d'investissement robustes, capables de naviguer dans la complexité des marchés modernes.

BIBLIOGRAPHIE & WEBOGRAPHIE

- ▶ OUVRAGES ET COURS
- ▶ ARTICLES SCIENTIFIQUES
- ▶ BIBLIOTHÈQUES PYTHON
- ▶ CODE SOURCE DU PROJET

Bibliographie et Webographie

Ouvrages et Cours de Référence

1. **Hafid, A.** (2026). *Chapter 2 : Linear Regression*. Cours de Machine Learning, Faculté des Sciences et Techniques d'Al Hoceima, Université Abdelmalek Essaâdi. Document pédagogique fourni par l'encadrant.
2. **Hafid, A.** (2026). *Chapter 3 : Logistic Regression*. Cours de Machine Learning, Faculté des Sciences et Techniques d'Al Hoceima, Université Abdelmalek Essaâdi. Document pédagogique fourni par l'encadrant.
3. **Géron, A.** (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3^e édition). O'Reilly Media. ISBN : 978-1-098-12597-4.
4. **Hull, J. C.** (2022). *Options, Futures, and Other Derivatives* (11^e édition). Pearson Education. ISBN : 978-0-13-628014-5.
5. **James, G., Witten, D., Hastie, T., & Tibshirani, R.** (2021). *An Introduction to Statistical Learning with Applications in R* (2^e édition). Springer. Disponible gratuitement : <https://www.statlearning.com>

Articles Scientifiques Fondateurs

6. **Engle, R. F.** (1982). Autoregressive Conditional Heteroscedasticity with Estimates of the Variance of United Kingdom Inflation. *Econometrica*, 50(4), 987–1007. <https://doi.org/10.2307/1912773>
7. **Bollerslev, T.** (1986). Generalized Autoregressive Conditional Heteroskedasticity. *Journal of Econometrics*, 31(3), 307–327. [https://doi.org/10.1016/0304-4076\(86\)90063-1](https://doi.org/10.1016/0304-4076(86)90063-1)
8. **Breiman, L.** (2001). Random Forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>

9. **Hoerl, A. E., & Kennard, R. W.** (1970). Ridge Regression : Biased Estimation for Nonorthogonal Problems. *Technometrics*, 12(1), 55–67. <https://doi.org/10.1080/00401706.1970.10488634>
10. **Wilder, J. W.** (1978). *New Concepts in Technical Trading Systems*. Trend Research.
(Ouvrage original introduisant l'indicateur RSI sur 14 jours.)
11. **Markowitz, H.** (1952). Portfolio Selection. *The Journal of Finance*, 7(1), 77–91. <https://doi.org/10.2307/2975974>

Documentation des Bibliothèques Python Utilisées

12. **Pedregosa, F. et al.** (2011). Scikit-learn : Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
Documentation officielle : <https://scikit-learn.org/stable/>
13. **Aroussi, R.** (2019–2024). *yfinance* : Yahoo! Finance market data downloader. Bibliothèque Python open-source.
<https://pypi.org/project/yfinance/>
Dépôt GitHub : <https://github.com/ranaroussi/yfinance>
14. **McKinney, W.** (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 51–56.
Documentation pandas : <https://pandas.pydata.org/docs/>
15. **Harris, C. R. et al.** (2020). Array programming with NumPy. *Nature*, 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
Documentation numpy : <https://numpy.org/doc/>
16. **Hunter, J. D.** (2007). Matplotlib : A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
Documentation matplotlib : <https://matplotlib.org/stable/>

Code Source du Projet

17. **EL Habnoui, S.** (2026). *Prévision de la Volatilité et Allocation Dynamique de Portefeuille par Apprentissage Automatique* — Code source Python complet du projet (Jupyter Notebook/ VS studio).
Dépôt GitHub : <https://github.com/safwanov/Projet-machine-learning>